

ALMA MATER STUDIORUM · UNIVERSITÀ DI BOLOGNA

TODO Dipartimento di Informatica — Scienza e Ingegneria
Corso di Laurea Triennale in Ingegneria e Scienze informatiche

TODO

TODO

Relatore:
Chiar.ma Prof.
TODO

Presentata da:
TODO

Correlatore:
Chiar.mo Prof.
TODO

Sessione MESE ANNO
Anno Accademico 20TODO/20xx

Indice

1	Introduzione	6
1.1	Iot (Internet of Things)	6
1.2	Publish/Subscribe	6
1.3	La Difficoltà del Percorso Ottimo	7
1.3.1	Grafo	7
1.3.2	Centralizzazione	8
1.4	Problema di Ottimizzazione	9
1.4.1	Rilassamento Continuo	9
1.4.2	Rilassamento Lagrangiano	9
1.4.3	Formulazione Matematica del Problema	14
1.5	Ambiente di Sviluppo	14
2	Descrizione del Problema e delle Formulazioni	16
2.1	Scenario del Problema	16
2.2	Simboli Globali	17
2.3	Prima Formulazione	18
2.4	Seconda Formulazione	19
2.5	Rilassamento Lagrangiano della Seconda Formulazione	21
2.5.1	Aggiornamento delle Penalità Lagrangiane	24
3	Miglioramento della Formulazione Rispetto ai Percorsi Parziali	25
3.1	Nuova Formulazione	25
3.2	Mancata Convergenza del Lower Bound alla Soluzione Intera	27
3.2.1	Analisi del Problema nel Dettaglio	27
3.2.2	Scrittura in Formulazione Forte	27
3.2.3	Nuove Condizioni di Attivazione	28
3.2.4	Terza Formulazione Completa	30
4	Implementazione Algoritmica	31
4.1	Implementazione delle Formulazioni Senza Soluzioni Parziali	31
4.1.1	Upper Bound Euristico	31
4.1.2	Prima Formulazione	33
4.1.3	Seconda Formulazione	36
4.2	Formulazione Con Soluzioni Parziali	42
4.2.1	Dijkstra a Scenari e Meccanismo di Discesa dell'Upper Bound	42
4.2.2	Terza formulazione	47
5	Analisi dei Risultati	53
5.1	Scenari e Test	53
5.1.1	Primo Scenario	53

5.1.2	Secondo Scenario	55
5.1.3	Terzo Scenario	57
5.1.4	Quarto Scenario	59
5.1.5	Caso 5	59
5.1.6	Caso 6	59
5.2	Analisi generale con grafici istogrammi e tabella con l'ottimo per ogni algoritmo per ogni scenario	59
5.3	Mostrare come cambia la convergenza al cambiare di alpha	59
5.4	Mostrare come cambia più soluzioni euristiche andiamo a prendere, cambiamo i valori dell'euristico	59
5.5	Soluzioni parziali	59
5.5.1	Caso 1	59
5.6	Caso rotto e ricalcolo	60
5.7	caso x	61
5.8	Modifica di alpha	61
5.9	Modifica di variabile x	61
6	Conclusioni	62

Sommario

TODO IoT (Internet of Things) è l'acronimo utilizzato per indicare la sfera tecnologica che ha alla base l'utilizzo di oggetti intelligenti. Per oggetti intelligenti si intendono quei dispositivi, macchine o attrezzature che si possono connettere a internet per trasmettere, ricevere dati o elaborarli. All'interno di questa sfera, uno dei metodi di comunicazione più utilizzati è quello basato su un modello Publish/Subscribe che consente lo scambio di dati tra dispositivi della rete senza la necessità di connessioni dirette tra loro. Per funzionare si utilizza il protocollo MQTT, uno dei protocolli più diffusi per lo scambio di messaggi, il quale si basa su una struttura con 3 tipologie di dispositivi connessi: Publisher, Subscriber e Broker. I Publisher una volta ottenuti dei dati tramite sensori di vario genere, li pubblica, rendendoli disponibili nella rete. I dati per spostarsi all'interno della rete utilizzano i broker cioè vari punti/server di connessione, i quali permettono il passaggio di dati per poi infine arrivare ai Subscribers per la lettura e l'elaborazione. Questa tesi riprende il lavoro effettuato da un collega nell'anno 21-22 approfondendo e sviluppando i casi di test con particolare attenzione a quelle che sono le caratteristiche che possono portare criticità al calcolo della soluzione ottima di percorso per il flusso dei dati. La tesi del collega si basa sull'idea di costruire un sistema distribuito sfruttando le proprietà matematiche del rilassamento Lagrangiano, dove ogni nodo conosce solamente i propri vicini.

1 Introduzione

1.1 Iot (Internet of Things)

Il termine Iot nasce come acronimo per indicare quella sfera tecnologica moderna che si basa sull'utilizzo di dispositivi "intelligenti". Quest'ultima ha avuto particolare rilevanza nell'ultimo periodo storico, andando a rivoluzionare il modo in cui si raccolgono e si processano le informazioni sia in ambito domestico sia industriale.

Un dispositivo definito intelligente o più comunemente chiamato "smart" è un dispositivo in grado di connettersi alla rete per poter svolgere diverse funzioni in cui tra le più importanti è presente lo scambio di informazioni.

In questa tesi ci si sofferma particolarmente sui "sensori", dispositivi chiave per lo scambio di messaggi in una rete IoT. Un sensore è un dispositivo che rileva un cambiamento nel mondo fisico, lo traduce in un segnale digitale e poi lo diffonde ad altri dispositivi, i quali lo utilizzeranno per prendere decisioni.

Una possibile criticità risiede proprio in questa parte del funzionamento, se una decisione deve essere presa con tempestività, anche la rete di trasmissione dovrà essere veloce e efficiente.

1.2 Publish/Subscribe

Tra le metodologie più famose di trasmissione di messaggi, quella che si intende approfondire è il Publish/Subscribe, un paradigma particolarmente adatto all'IoT, nato come sostituzione del classico Client-Server. In questo schema i dispositivi si dividono in Publisher e Subscriber collegati tra loro tramite dei nodi/server di trasmissione definiti Broker.

In questo modello ognuno ha il suo ruolo: i Publisher devono diffondere nella rete le informazioni in loro possesso, i Broker devono ritrasmettere queste informazioni senza permettere danneggiamenti o perdita delle stesse e i Subscriber sono i ricevitori o i richiedenti. Per quanto riguarda il meccanismo alla base è molto semplice: il Publisher invia messaggi etichettati con un topic, il Broker li riceve e li inoltra solamente ai Subscriber iscritti a quel topic.

Questo approccio permette un disaccoppiamento tra nodo di partenza e di arrivo, in quanto le due parti non si conoscono direttamente e non devono per forza essere attive contemporaneamente per garantire il successo della comunicazione.

Il protocollo standard per l'implementazione di questo modello è l'MQTT, progettato per essere leggero e robusto anche su reti instabili ma che presenta alla base una criticità importante: essendo il Broker il nodo centrale di comunicazione, la sua irraggiungibilità diventa rischio di interruzione per l'intera rete. Per ovviare al problema, si è optato per la creazione di infrastrutture con più Broker configurati per lavorare e apparire come uno solo, in modo che nel caso di fallimento di un server, i rimanenti possano continuare il lavoro.

Facendo riferimento al paragrafo precedente si può associare il ruolo di Publisher

ai sensori e il ruolo di Subscriber a tutti quei dispositivi che necessitano del valore rilevato.

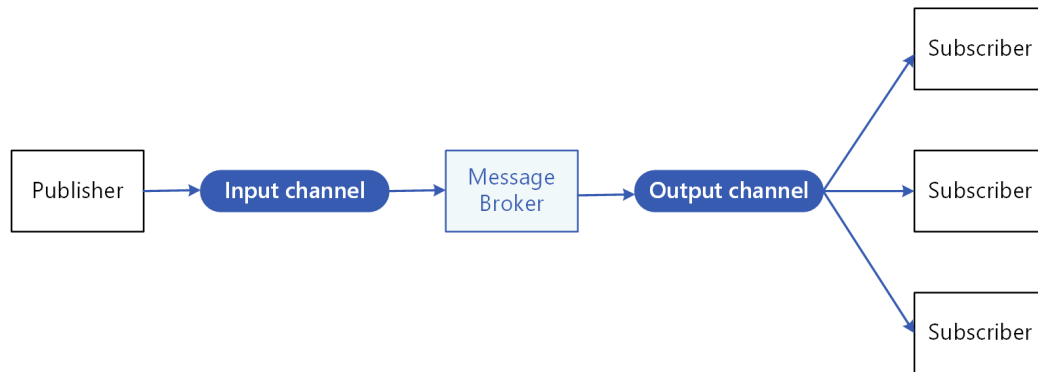


Figura 1.1: Immagine di funzionamento di una rete Publish/Subscribe

1.3 La Difficoltà del Percorso Ottimo

In un sistema moderno, per garantire la velocità di una rete è cruciale trovare il percorso ottimo per la trasmissione dei dati, infatti una scelta non ottimizzata potrebbe ritardare o addirittura impedire la ricezione delle informazioni, compromettendo il funzionamento dei dispositivi che dipendono da esse. Per questo motivo è necessario individuare il metodo più efficiente per rappresentare la rete e per calcolare il percorso ottimale.

1.3.1 Grafo

Uno dei metodi migliori per poter descrivere una rete di dispositivi connessi è tramite un grafo. Usando questa struttura, i dispositivi vengono rappresentati tramite dei nodi e i collegamenti tramite degli archi.

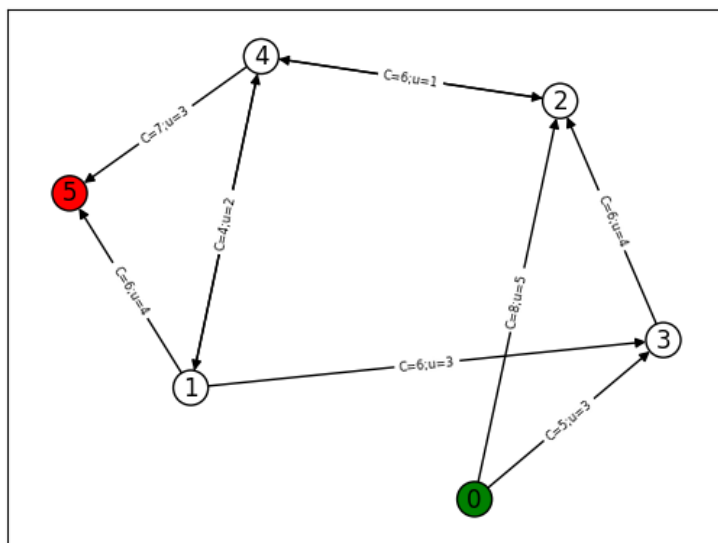


Figura 1.2: Immagine di un grafo con costi e capacità degli archi

L'uso di un grafo risulta una scelta efficiente perché, per reti semplici, consente di mostrare in modo naturale sia i componenti della rete sia le relazioni tra essi e grazie alla sua flessibilità permette inoltre di aggiungere varie proprietà agli elementi rappresentati, rendendo possibile modellare scenari complessi con facilità.

1.3.2 Centralizzazione

Un altro aspetto fondamentale del problema riguarda la centralizzazione dell'elaborazione della rete. In un caso reale, calcolare qual è il percorso ottimo necessita di una notevole potenza di calcolo e, soprattutto, bisogna che ci sia la garanzia di possedere informazioni complete sulla composizione della rete. Da qui nasce l'idea di non affidarsi più esclusivamente ad un server centrale ma di distribuire il calcolo su tutti i dispositivi della rete rendendola autonoma sia nella ricerca del percorso ottimo sia nella gestione dei guasti in caso di malfunzionamento di un canale.

Peer-To-Peer

Ciò è reso possibile sfruttando un protocollo Peer-to-Peer, ovvero un protocollo che tratta i dispositivi in comunicazione sia come Client sia come Server, permettendogli dunque di scambiarsi richieste di risorse o servizi senza dover passare da terzi.

1.4 Problema di Ottimizzazione

Il problema descritto è così definito come un problema di ottimizzazione, problema che richiede di trovare la migliore soluzione possibile fra tutte le soluzioni ammissibili, massimizzando o minimizzando una funzione definita obiettivo e rispettando i vincoli imposti.

Questo tipo di problema è descrivibile utilizzando un modello matematico:

$$\min/\max \sum_i c_i x_i \quad (1.1)$$

vincoli:

$$Ax \geq b \quad (1.2)$$

$$Bx \geq d \quad (1.3)$$

$$x \in \{0, 1\} \quad (1.4)$$

Con c il vettore dei costi, x la variabile decisionale, A e B le matrici dei coefficienti del problema, b e d i vettori delle costanti per i vincoli.

Molto spesso quando si tratta di problemi di questo tipo, si possono riscontrare delle difficoltà nella risoluzione, per questo viene adottata la tecnica del rilassamento dei vincoli che consente di semplificare il calcolo ottenendo un limite inferiore per la soluzione del problema originale.

1.4.1 Rilassamento Continuo

Il rilassamento più intuitivo è quello continuo, dove se il problema presenta la variabile decisionale come intera, è possibile costruire un problema di ottimizzazione equivalente, abbandonando il vincolo di interezza della variabile e permettendole di assumere un qualsiasi valore reale all'interno di un intervallo continuo derivato dal vincolo originale.

Questo approccio trasforma il problema intero in un problema di programmazione lineare continua, per il quale esistono algoritmi efficienti. Lo spazio delle soluzioni ammissibili, tuttavia, è più ampio e il valore ottimo del problema rilassato potrebbe non essere sempre applicabile al problema originale.

$$x \in \{0, 1\} \rightarrow 0 \leq x \leq 1 \quad (1.5)$$

1.4.2 Rilassamento Lagrangiano

Un rilassamento particolarmente utile è quello lagrangiano, che permette di rilassare i vincoli computazionalmente dispendiosi aggiungendoli alla funzione obiettivo e associando a ognuno di essi un moltiplicatore di Lagrange. Quest'ultimo

funge da penalità: più ci si allontana dal rispetto del vincolo, maggiore sarà il peggioramento del valore della funzione obiettivo, allontanandolo dall'ottimo. Il risultato è una nuova funzione obiettivo, detta lagrangiana, che dipende sia dalle variabili originali del problema sia dai moltiplicatori di Lagrange, e la cui risoluzione fornirà i valori ottimi di entrambi.

Spiegazione Matematica

Si considera un problema di minimizzazione P:

$$z(P) = \min cx \quad (1.6)$$

vincoli:

$$Ax \geq b \quad (1.7)$$

$$x \in \{0, 1\} \quad (1.8)$$

Con c il vettore dei costi, x la variabile decisionale, A la matrice dei coefficienti del problema, b il vettore delle costanti per il vincolo.

Con il rilassamento lagrangiano si rilassa il vincolo $Ax \geq b$ aggiungendo una penalità λ non negativa e inserendolo all'interno della funzione obiettivo.

$$z(P) = \min cx, Ax - b \geq 0 \quad (1.9)$$

↓

$$L(\lambda) = \min cx - \lambda(Ax - b) \quad (1.10)$$

vincoli:

$$x \in \{0, 1\} \quad (1.11)$$

Il motivo del segno $-$ è dovuto al fatto che il vincolo è soddisfatto quando assume un valore maggiore o uguale a 0, quindi dato che λ è definita non negativa, se il vincolo non viene soddisfatto come nel caso in cui sia negativo, sottrarlo dalla funzione obiettivo ne aumenta il valore, allontanandolo dal minimo ricercato.

Il valore ottimale del problema originale $z(P)$ sarà maggiore o al massimo uguale al valore ottimale del problema rilassato $L(\lambda)$.

$$z(P) \geq L(\lambda), \quad \forall \lambda \geq 0 \quad (1.12)$$

Questo perché non avendo l'obbligo di rispettare il vincolo, che è stato rilassato, lo spazio delle soluzioni ammissibili è più ampio e quindi si possono trovare soluzioni che il problema originale non può considerare corrette, le quali, avendo meno costrizioni, avranno un valore più basso o al massimo uguale alla soluzione dell'originale.

1. Si prende una x^* che è soluzione ottima di $z(P)$ quindi è ammissibile e il vincolo è rispettato.
2. Per $\lambda \geq 0$ e $Ax^* - b \geq 0$ si ha:

$$cx^* \geq cx^* - \lambda(ax^* - b) \quad (1.13)$$

3. x^* è anche soluzione ammissibile per il problema rilassato quindi esiste sicuramente una soluzione di $L(\lambda)$ che sarà o minore o uguale al valore di x^* :

$$L(\lambda) = \min(cx - \lambda(Ax - b)) \leq cx^* - \lambda(Ax^* - b) \quad (1.14)$$

4. Unendo le disuguaglianze:

$$z(P) = cx^* \geq cx^* - \lambda(ax^* - b) \geq L(\lambda) \rightarrow z(P) \geq L(\lambda) \quad (1.15)$$

Grazie a questa proprietà, si può pensare di massimizzare la funzione lagrangiana per ottenere il migliore limite inferiore possibile per il problema originale e si può descrivere con la formula del duale lagrangiano:

$$z(D_L) = \max_{\lambda \geq 0} [L(\lambda)] \quad (1.16)$$

Una delle tecniche che può essere usata per la ricerca della soluzione del duale lagrangiano è il metodo del subgradiente.

Metodo del Subgradiente

Il metodo del subgradiente è un metodo iterativo usato per massimizzare una funzione concava, come quella lagrangiana, con lo scopo di ottenere il migliore limite inferiore per il problema originale. Nello specifico, si può utilizzare per calcolare i moltiplicatori di Lagrange associati ai vincoli rilassati che forniscono il risultato $L(\lambda)$ più alto.

La funzione lagrangiana non è perfettamente una curva liscia ma è composta di segmenti, ciascuno corrispondente a una soluzione ottima $\bar{x}$ per il λ specificato.

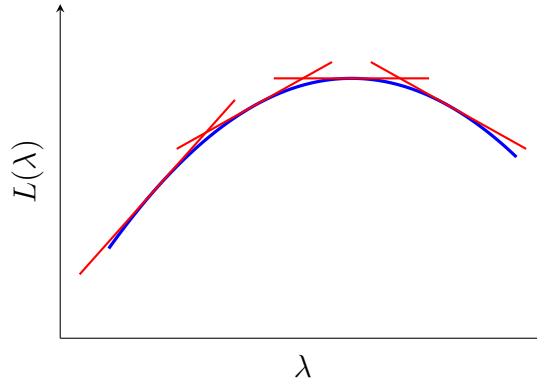


Figura 1.3: Immagine di una funzione lagrangiana con inviluppo

Data una λ , si ottiene una $\bar{x}$ tale che:

$$L(\bar{\lambda}) = \min(cx - \bar{\lambda}(Ax - b)) = c\bar{x} - \bar{\lambda}(A\bar{x} - b) \quad (1.17)$$

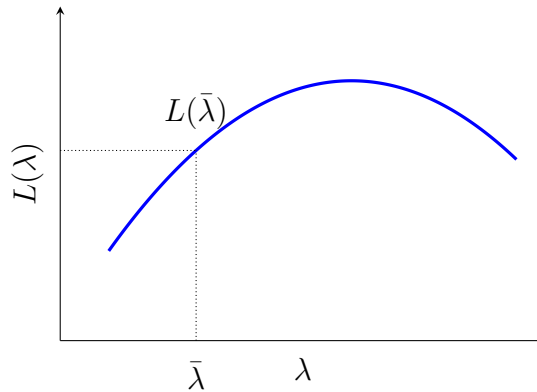


Figura 1.4: Valore di $L(\bar{\lambda})$ corrispondente a una $\bar{\lambda}$ scelta

Riprendendo la soluzione $\bar{x}$ derivata dalla $\bar{\lambda}$ data, si può ridefinire $L(\lambda)$, cioè il valore ottimo in una qualsiasi altra λ generica, come o minore o uguale della soluzione trovata ora:

$$L(\lambda) = \min(cx - \lambda(Ax - b)) \leq c\bar{x} - \lambda(A\bar{x} - b) \quad (1.18)$$

Questo perché la disequazione confronta elementi diversi, nel lato sinistro è presente una x ottima per la nuova λ , mentre nel lato destro è presente la $\bar{x}$, ottima per $\bar{\lambda}$ ma non necessariamente per λ . Poiché le si stanno valutando con la stessa λ , allora per definizione, la x ottima darà un valore minore o, nel caso in cui anche $\bar{x}$

sia ottima per λ , uguale.

Sottraendo ora l'equazione (1.17) dalla (1.18) si ottiene:

$$L(\lambda) - L(\bar{\lambda}) \leq c\bar{x} - \lambda(A\bar{x} - b) - (c\bar{x} - \bar{\lambda}(A\bar{x} - b)) \quad (1.19)$$

$\downarrow$

$$L(\lambda) - L(\bar{\lambda}) \leq -\lambda(A\bar{x} - b) + \bar{\lambda}(A\bar{x} - b) \quad (1.20)$$

$\downarrow$

$$L(\lambda) - L(\bar{\lambda}) \leq (A\bar{x} - b)(-\lambda + \bar{\lambda}) \quad (1.21)$$

$\downarrow$

$$L(\lambda) \leq L(\bar{\lambda}) + (A\bar{x} - b)(-\lambda + \bar{\lambda}) \quad (1.22)$$

Si definisce $s = -(A\bar{x} - b)$ e si trasforma l'equazione:

$$L(\lambda) \leq L(\bar{\lambda}) + s(\lambda - \bar{\lambda}) \quad (1.23)$$

L'obiettivo della risoluzione del duale lagrangiano è, a ogni iterazione, di far crescere $L(\lambda)$ fino a trovare il valore massimo. In termini matematici si vuole che $L(\lambda)$ sia maggiore di $L(\bar{\lambda})$ e che quindi la loro differenza sia positiva. L'unico modo di ottenere questo risultato è di avere il termine: $s(\lambda - \bar{\lambda}) > 0$ e che quindi sommandolo a $L(\bar{\lambda})$ ne aumenti il valore. Per garantire che ciò accada si opta per spostare λ rispetto a $\bar{\lambda}$ usando lo stesso verso del subgradiente s in modo che:

$$\lambda = \bar{\lambda} + \theta s \quad (1.24)$$

$\downarrow$

$$\lambda - \bar{\lambda} = \theta s \quad (1.25)$$

$$s(\lambda - \bar{\lambda}) = s(\theta s) = \theta s^2 \quad (1.26)$$

Di conseguenza per un $s \neq 0$ e per un passo $\theta > 0$ il valore risultante sarà sicuramente positivo e porterà ad una λ maggiore rispetto a $\bar{\lambda}$.

Anche la scelta del parametro θ è fondamentale, un passo troppo grande potrebbe far saltare la soluzione ottima portando alla divergenza, mentre un passo troppo corto potrebbe convergere alla soluzione ottima troppo lentamente.

1.4.3 Formulazione Matematica del Problema

Nei problemi di ottimizzazione, uno degli aspetti più cruciali risiede nel modo in cui le relazioni tra le variabili vengono tradotte in vincoli. Uno stesso problema può essere modellato attraverso diversi insiemi di vincoli, ciascuno dei quali si definisce come una formulazione del problema.

Due formulazioni si definiscono equivalenti se possiedono lo stesso identico insieme di soluzioni ammissibili intere. Tuttavia, nonostante l'equivalenza sull'insieme delle soluzioni, le loro prestazioni computazionali possono differire di molto, rendendo la scelta della formulazione un fattore chiave per la qualità del modello matematico creato.

Debole Contro Forte

I risolutori, nel cercare la soluzione ottima, non operano direttamente con le variabili intere, ma applicano il rilassamento continuo permettendo loro di assumere valori frazionari. È proprio durante questa fase di calcolo che si distinguono due tipologie di formulazioni diverse:

- Formulazione debole: Dopo aver effettuato il rilassamento, il risolutore è libero di assegnare alle variabili valori frazionari molto distanti dai limiti interi reali. Di conseguenza il modello può ottenere un valore ottimo rilassato anche molto distante dall'ottimo intero.
- Formulazione forte: I vincoli sono scritti in modo tale da limitare pesantemente le scelte frazionarie favorendo l'ottenimento di valori interi per le variabili e di conseguenza, un valore ottimo molto vicino all'ottimo intero se non coincidente.

1.5 Ambiente di Sviluppo

Esistono molti ambienti di sviluppo che permettono la risoluzione di problemi di ottimizzazione, ma quelli su cui ci si vuole soffermare in questa tesi sono CPLEX Optimization Studio e Google Colab.

CPLEX Optimization Studio è un software sviluppato da IBM con l'obiettivo di permettere la risoluzione di problemi di ottimizzazione in maniera intuitiva senza l'obbligo di dover implementare i metodi risolutivi da parte dell'utente finale. Per garantire semplicità è richiesta solamente la struttura del problema, i dati e la configurazione di avvio. È un software potente ma presenta delle limitazioni: è particolarmente costoso in termini economici, usa un linguaggio di programmazione molto specifico, non facilmente riconoscibile a prima vista da un utente

inesperto e modellare problemi di rilassamento lagrangiano al suo interno è complesso.

Google Colab invece è un ambiente Python online scelto per diversi vantaggi: le librerie sono già installate o facilmente installabili, i file sono salvati nel cloud, quindi accessibili da più dispositivi e la modalità a celle permette di eseguire porzioni di codice senza dover rieseguire tutto ogni volta. La scelta di Python è dovuta sia a delle difficoltà tecniche nel far collaborare l'ambiente di sviluppo CPLEX con Java, sia al fatto che si parla di un ambiente estremamente flessibile e in quanto tale, permette di importare numerose librerie per la semplificazione dei calcoli. È grazie a questa flessibilità che l'ambiente di Google Colab permette di modellare anche problemi di ottimizzazione complessi, come il rilassamento lagrangiano, e permette la visualizzazione delle casistiche tramite librerie grafiche come Networkx e Matplotlib.pyplot.

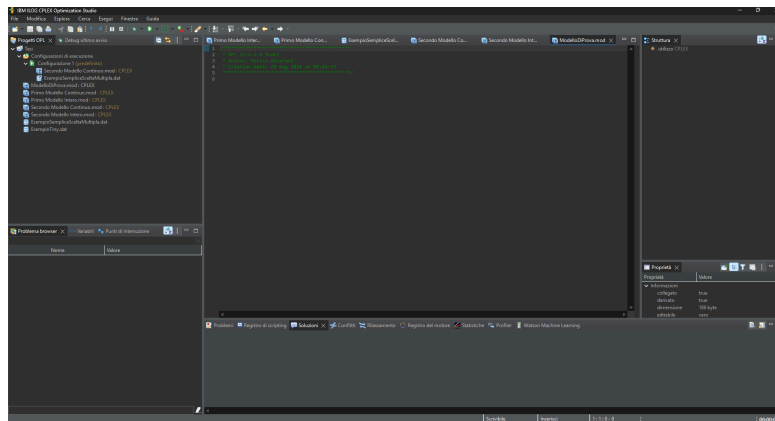


Figura 1.5: Immagine dell'ambiente di sviluppo di CPLEX Optimization Studio

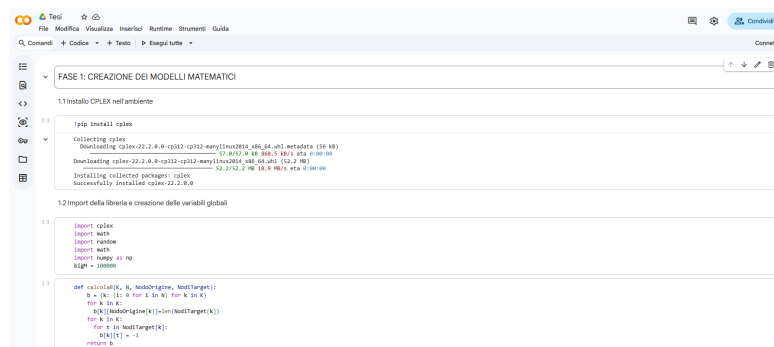


Figura 1.6: Immagine dell'ambiente di sviluppo di Google Colab

2 Descrizione del Problema e delle Formulazioni

2.1 Scenario del Problema

Il problema che si vuole approfondire nello specifico è quello di una rete IoT in cui si vogliono scambiare informazioni tra generatori di dati e ricevitori. La rete è caratterizzata da una struttura Publish/Subscribe che, come definita in precedenza, presenta al suo interno dei nodi di comunicazione definiti Broker, sensori che svolgono il ruolo di generatori dei dati (Publisher) e utenti finali che svolgono il ruolo di ricevitori (Subscriber).

L'obiettivo della tesi è sviluppare, a partire dal lavoro di un collega, un algoritmo efficiente per il calcolo del costo del percorso ottimale. L'algoritmo dovrà essere in grado di gestire sia il calcolo di percorsi parziali in presenza di destinatari non raggiungibili, sia scenari di multicasting, ovvero situazioni in cui la stessa informazione è richiesta contemporaneamente da più utenti.

Per modellare correttamente il problema è necessario considerare un caso reale, quindi con diversi fattori in gioco:

- Capacità del canale (quante informazioni possono passare insieme nello stesso collegamento nello stesso momento).
- Costo del canale (quanto è dispendioso in termini economici o computazionali utilizzare quel determinato percorso).
- Peso di un'informazione (quanta banda occupa la singola informazione all'interno del canale).

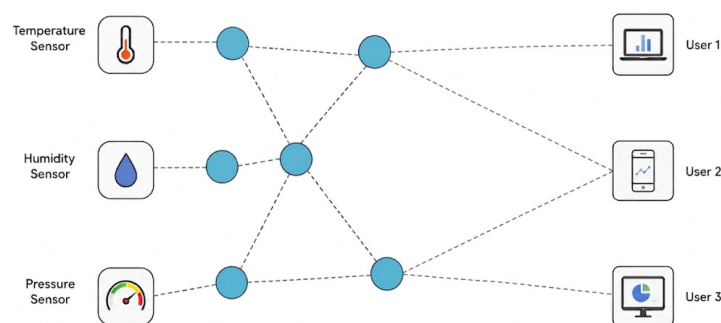


Figura 2.1: Immagine di una rete con dei Publisher (sensori), dei Broker e dei Subscriber (utenti finali)

2.2 Simboli Globali

Per poter iniziare una transizione del problema dal mondo reale a una formulazione matematica è necessario mappare le variabili reali in un contesto informatico/matematico:

- K , è l'insieme contenente le informazioni da inviare o ricevere.
- N , è l'insieme contenente tutti i nodi della rete.
- *Publisher*, è l'insieme contenente i nodi che secondo il modello Publish/Subscribe fanno parte dei generatori delle informazioni.
- *Broker*, è l'insieme contenente i nodi di mezzo che non devono nè generare nè consumare le informazioni ma solo ritrasmettere.
- *Subscriber*, è l'insieme contenente i nodi che secondo il modello Publish/Subscribe devono ricevere le informazioni.
- A , è l'insieme contenente gli archi della rete quindi i canali di comunicazione.
- $NodoOrigine_k$, è l'insieme contenente per ogni informazione k il suo nodo di origine.
- $NodiTarget_k$, è l'insieme contenente per ogni informazione k i nodi che richiedono quella informazione.
- b_i^k , è una variabile che, a seconda del nodo i e della informazione k , indica il ruolo di i rispetto a k : se è maggiore di 0 allora i è un generatore e produce b_i^k informazioni k , se è uguale a 0 allora i è un nodo di trasmissione e invece se è uguale a -1 allora i è un nodo richiedente di k .
- a_k , è una variabile che indica per ogni informazione k quanta banda occupa all'interno di un canale.
- c_{ij} , è l'insieme contenente per ogni arco i, j il suo costo.
- u_{ij} , è l'insieme contenente per ogni arco i, j la sua capacità.
- Lt_i , è l'insieme contenente per ogni nodo i tutti i nodi raggiungibili da i .
- Lr_i , è l'insieme contenente per ogni nodo i tutti i nodi da cui si può raggiungere i .

- x_{ij}^k , è una variabile decisionale che indica quanti Subscriber, richiedenti l'informazione k , stanno venendo serviti usando l'arco i, j .
- e_{ij}^k , è una variabile decisionale che indica se per il trasporto di una informazione k l'arco i, j è attivo.
- M , è una variabile di valore molto alto, solitamente pari al numero dei nodi, usata per definire dei vincoli.

2.3 Prima Formulazione

Una prima formulazione può essere descritta con uno schema minimum-cost maximum-flow, un problema comunemente noto nell'ambito dell'ottimizzazione dove viene richiesto di soddisfare il maggior numero di richieste possibili minimizzando i costi nel farlo.

Funzione Obiettivo:

$$\text{minimizza } \sum_{k \in K} \sum_{i \in N} \sum_{j \in Lr_i} c_{ji} x_{ji}^k \quad (2.1)$$

Vincoli:

$$\sum_{j \in Lr_i} x_{jt}^k = 1 \quad \forall t \in NodiTarget_k, k \in K \quad (2.2)$$

$$\sum_{j \in Lr_i} x_{ji}^k = \sum_{j \in Lt_i} x_{ij}^k \quad \forall i \in Broker, k \in K \quad (2.3)$$

$$M e_{ij}^k \geq x_{ij}^k \quad \forall (i, j) \in A, k \in K \quad (2.4)$$

$$\sum_{k \in K} a^k (e_{ij}^k + e_{ji}^k) \leq u_{ij} \quad \forall i \in N, j \in Lt_i \quad (2.5)$$

$$x_{ij}^k \geq 0 \quad \forall (i, j) \in A, k \in K \quad (2.6)$$

$$e_{ij}^k \in \{0, 1\} \text{ oppure } 0 \leq e_{ij}^k \leq 1 \quad \forall (i, j) \in A, k \in K \quad (2.7)$$

Alla base di questa formulazione si immagina che ogni copia dell'informazione abbia un costo e che, quindi, per ogni Subscriber soddisfatto aumenti la spesa totale, ma poiché la copia del messaggio viene generata solo dopo il suo transito, la

capacità prende in considerazione solo se l'arco è attivo o meno. Questa formulazione nasce con l'idea di un approccio centralizzato e richiede perciò un computer centrale per il calcolo dell'ottimo.

(2.1): Per ogni informazione k , per ogni nodo i , si somma il costo degli archi entranti in i moltiplicati per il numero di Subscriber serviti su quell'arco. In pratica il costo totale dipende da quanti Subscriber vengono serviti per ogni informazione k e da quali archi si sta utilizzando per farlo.

(2.2): Sommando le informazioni in entrata al Subscriber il risultato deve corrispondere a 1. Questo vincolo permette di garantire che ogni Subscriber riceva l'informazione che ha richiesto, né di meno né di più. Questo deve valere per ogni Subscriber e per ogni informazione richiesta.

(2.3): Sommando il numero di Subscriber serviti tramite gli archi entranti nel Broker i , il risultato deve essere uguale al numero di Subscriber serviti tramite gli archi uscenti dal medesimo nodo. Questo vincolo garantisce che i sia esclusivamente un nodo di passaggio e che non possa generare o consumare informazioni.

(2.4): Se si sta soddisfacendo un Subscriber tramite l'arco i, j allora quest'ultimo deve essere considerato attivo rispetto al transito dell'informazione k . Questo vincolo serve a legare la variabile e con la variabile x , garantendo che se si sta usando un arco allora esso è attivo. La scelta di M ricade sul numero totale dei nodi poiché x , potendo assumere al massimo un valore pari al numero di nodi Subscriber, non potrà mai superare tale soglia.

(2.5): Per ogni arco i, j si controlla che la somma della banda occupata da tutte le informazioni passanti in qualsiasi direzione non sia mai maggiore della capacità totale. Questo vincolo garantisce di non superare mai il valore massimo della capacità di un arco.

(2.6): Per ogni arco i, j e per ogni informazione k , la variabile x deve essere non negativa in quanto rappresenta il numero di Subscriber serviti tramite quell'arco e per quella informazione.

(2.7): Per ogni arco i, j e per ogni informazione k , la variabile e indica se l'arco è attivo per il transito dell'informazione. Nel caso di modello intero assume valore binario mentre nel caso di rilassamento continuo può assumere qualsiasi valore reale nell'intervallo $[0, 1]$.

2.4 Seconda Formulazione

Una seconda formulazione del problema si può basare sull'idea che la generazione dell'informazione sia priva di costo e, quindi, che la spesa totale sia associata esclusivamente all'attivazione degli archi. Un obiettivo raggiungibile con questa formulazione è quello di essere facilmente rilassabile con il metodo lagrangiano,

permettendo in questo modo di creare sotto-problemi locali, ovvero problemi dipendenti esclusivamente dal nodo su cui sono definiti insieme agli archi e ai nodi adiacenti.

Funzione Obiettivo:

$$\text{minimizza} \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k \quad (2.8)$$

Vincoli:

$$\sum_{j \in Lt_i} x_{ij}^k - \sum_{j \in Lr_i} x_{ji}^k = b_i^k \quad \forall i \in N - \{NodoOrigine_k\}, k \in K \quad (2.9)$$

$$\sum_{k \in K} a^k e_{ij}^k \leq u_{ij} \quad \forall (i, j) \in A \quad (2.10)$$

$$e_{ij}^k \leq x_{ij}^k \leq e_{ij}^k |NodiTarget_k| \quad \forall (i, j) \in A, k \in K \quad (2.11)$$

$$x_{ij}^k \geq 0 \quad \forall (i, j) \in A, k \in K \quad (2.12)$$

$$e_{ij}^k \in \{0, 1\} \text{ oppure } 0 \leq e_{ij}^k \leq 1 \quad \forall (i, j) \in A, k \in K \quad (2.13)$$

Questa formulazione nasce con l'idea di un possibile approccio decentralizzato e rende possibile perciò effettuare il calcolo dell'ottimo all'interno dei nodi distribuiti.

(2.8): Per ogni informazione k , si somma il costo dell'arco per la sua attivazione rispetto al trasporto di k . Minimizzando si cerca di usare il minor numero di archi possibili soddisfacendo comunque tutte le richieste.

(2.9): Per ogni nodo i e per ogni informazione k , la differenza tra le informazioni uscenti e entranti deve essere uguale a b . Questo vincolo garantisce che ogni nodo svolga correttamente il proprio ruolo all'interno della rete per ogni informazione considerata. La creazione di questo vincolo nasce con l'intenzione di unire le condizioni (2.2) e (2.3) della prima formulazione in modo da avere un solo vincolo che analizza sia gli archi entranti sia quelli uscenti, semplificando il rilassamento lagrangiano.

(2.10): Per ogni arco i, j si verifica che la somma della banda occupata dalle informazioni passanti non sia mai maggiore della capacità massima. La differenza del vincolo rispetto alla versione della prima formulazione (2.5) risiede nella gestione delle capacità: anziché considerare ogni arco come singolo e bidirezionale

con una capacità condivisa tra le due direzioni, ora si considera come due archi distinti monodirezionali con capacità indipendenti. Questa scelta viene effettuata per semplificare il rilassamento lagrangiano, rendendo il vincolo dipendente dal singolo arco.

(2.11): Per ogni arco i, j e per ogni informazione k , il valore della variabile x è legato alla variabile e : se l'arco è disattivato nessuna informazione può transitarvi, altrimenti se è attivo il numero di informazioni può essere scelto in un intervallo da uno fino al numero di Subscriber richiedenti. Questo vincolo garantisce un legame tra l'attivazione di un arco e il numero di informazioni che passano al suo interno. I vincoli (2.12) e (2.13) sono analoghi ai vincoli (2.6) e (2.7).

2.5 Rilassamento Lagrangiano della Seconda Formulazione

La seconda formulazione, con le modifiche apportate ai vincoli rispetto alla prima, risulta facilmente rilassabile usando il metodo lagrangiano, permettendo dunque di ottenere una funzione obiettivo separabile in sotto-problemi indipendenti, uno per ogni arco della rete.

Prima di procedere col rilassamento si definiscono $k \times i$ penalità lagrangiane: λ_i^k con k informazione e i nodo. Il numero corrisponde a quanti vincoli sono stati rilassati e in questo caso corrisponde al vincolo (2.9) che deve essere rispettato per ogni possibile combinazione di i e k .

Si riscrive il vincolo nella forma di uguaglianza a zero:

$$\sum_{j \in Lt_i} x_{ij}^k - \sum_{j \in Lr_i} x_{ji}^k - b_i^k = 0 \quad (2.14)$$

Questo viene successivamente incorporato nella funzione obiettivo insieme alle penalità:

$$L(\lambda) = \min \sum_{k \in K} \sum_{i, j \in A} c_{i,j} e_{ij}^k + \sum_{k \in K} \sum_{i \in N} \lambda_i^k \left(\sum_{j \in Lt_i} x_{ij}^k - \sum_{j \in Lr_i} x_{ji}^k - b_i^k \right) \quad (2.15)$$

(2.15): La doppia sommatoria $\sum_{k \in K} \sum_{i \in N}$ è necessaria perché si vuole trovare il minimo della funzione considerando che il vincolo rilassato venga rispettato su tutte le combinazioni di k e i come da esso definito (2.9).

(2.15): Il $+$ prima delle sommatorie è una scelta di comodità. In caso di violazione del vincolo, il segno di λ si adatta al segno del termine $(\sum_{j \in Lt_i} x_{ij}^k - \sum_{j \in Lr_i} x_{ji}^k - b_i^k)$ usando il metodo del subgradiente. Ciò tenderà a rendere il prodotto tra i due positivo e allontanerà la soluzione dal minimo, penalizzando il mancato rispetto

del vincolo.

Si ha quindi:

$$L(\lambda) = \min \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + \sum_{k \in K} \sum_{i \in N} \lambda_i^k \sum_{j \in Lt_i} x_{ij}^k - \sum_{k \in K} \sum_{i \in N} \lambda_i^k \sum_{j \in Lr_i} x_{ji}^k - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k \quad (2.16)$$

↓

$$L(\lambda) = \min \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + \sum_{k \in K} \sum_{i,j \in A} \lambda_i^k x_{ij}^k - \sum_{k \in K} \sum_{j,i \in A} \lambda_i^k x_{ji}^k - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k \quad (2.17)$$

(2.16 $\rightarrow$ 2.17): La trasformazione delle sommatorie da $\sum_{k \in K} \sum_{i \in N} \lambda_i^k \sum_{j \in Lt_i} x_{ij}^k$ a $\sum_{k \in K} \sum_{i,j \in A} \lambda_i^k x_{ij}^k$ viene effettuata applicando prima la proprietà della linearità della sommatoria: $\sum_{k \in K} \sum_{i \in N} \sum_{j \in Lt_i} \lambda_i^k x_{ij}^k$ poi, notando che la doppia sommatoria $\sum_{i \in N} \sum_{j \in Lt_i}$ considera esattamente tutti gli archi i, j della rete, trascrivendola sostituendo $\sum_{i \in N} \sum_{j \in Lt_i}$ con $\sum_{i,j \in A}$.

Successivamente, il termine $\sum_{k \in K} \sum_{j,i \in A} \lambda_i^k x_{ji}^k$ viene rinominato cambiando $j \rightarrow i$ e $i \rightarrow j$:

$$\sum_{k \in K} \sum_{i,j \in A} \lambda_j^k x_{ij}^k \quad (2.18)$$

Ne consegue:

$$L(\lambda) = \min \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + \sum_{k \in K} \sum_{i,j \in A} \lambda_i^k x_{ij}^k - \sum_{k \in K} \sum_{i,j \in A} \lambda_j^k x_{ij}^k - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k \quad (2.19)$$

↓

$$L(\lambda) = \min \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + \sum_{k \in K} \sum_{i,j \in A} (\lambda_i^k - \lambda_j^k) x_{ij}^k - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k \quad (2.20)$$

↓

$$L(\lambda) = \min \sum_{k \in K} \sum_{i,j \in A} (c_{ij} e_{ij}^k + (\lambda_i^k - \lambda_j^k) x_{ij}^k) - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k \quad (2.21)$$

(2.19) $\rightarrow$ (2.21): Nel passaggio tra le equazioni sono state svolte due semplici operazioni: un raccoglimento dei termini comuni e l'applicazione della proprietà della

linearità.

Dato che l'ultimo termine non ha variabili decisionali può essere estratto in quanto nella ricerca del minimo non influenza il risultato.

$$L(\lambda) = \min \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + (\lambda_i^k - \lambda_j^k) x_{ij}^k \quad (2.22)$$

La funzione obiettivo risulta ora separabile: per ogni arco i, j si ottiene un sotto-problema indipendente definito LR .

$$LR_{i,j \in A}(\lambda) = \min \sum_{k \in K} c_{ij} e_{ij}^k + (\lambda_i^k - \lambda_j^k) x_{ij}^k \quad (2.23)$$

$$L(\lambda) = \sum_{i,j \in A} LR_{i,j}(\lambda) - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k \quad (2.24)$$

$LR_{i,j}(\lambda)$ indica la soluzione ottima del sotto-problema per l'arco su cui è definita. Per la sua risoluzione si introduce una variabile d_{ij}^k che quantifica il costo dell'attivazione dell'arco i, j per l'informazione k :

$$\min c_{ij} e_{ij}^k + (\lambda_i^k - \lambda_j^k) x_{ij}^k \quad (2.25)$$

Si presentano due casi: se l'arco è disattivato, il costo è pari a 0, poiché $e_{ij}^k = 0$ implica $x_{ij}^k = 0$ per il vincolo (2.11). Se invece è attivato, d_{ij}^k rappresenta il costo dell'attivazione. Nel caso in cui d_{ij}^k sia minore di 0 conviene attivare l'arco in quanto si ha un guadagno.

Per calcolare d_{ij}^k si hanno già tutti i valori necessari: c_{ij} è noto, si pone $e_{ij}^k = 1$ poiché si vuole confrontare se si ha un guadagno dall'attivazione dell'arco e x_{ij}^k dipende da $(\lambda_i^k - \lambda_j^k)$. Poiché l'obiettivo è minimizzare, se il termine $(\lambda_i^k - \lambda_j^k)$ fosse positivo allontanerebbe dalla soluzione ottima e quindi x_{ij}^k assume il valore minimo consentito dal vincolo (2.11). Se invece fosse negativo contribuirebbe alla minimizzazione e quindi x_{ij}^k assume il valore massimo consentito che è pari a $NodiTarget_k$.

Determinati quali archi attivare e per quali informazioni, si incontra un ulteriore problema legato alla capacità degli archi: non tutte le informazioni selezionate potrebbero essere trasmesse simultaneamente all'interno dell'arco senza superare la sua capacità massima. Questo problema è riconducibile al classico problema del Knapsack in cui l'arco rappresenta lo zaino con capacità u_{ij} e le informazioni k sono gli oggetti con peso a_k e valore $-d_{ij}^k$ che si vuole inserire all'interno. Poiché il Knapsack massimizza il valore degli oggetti inseriti e il mio guadagno è rappresentato in negativo da d_{ij}^k , invertendone il segno rendo compatibili i due problemi.

2.5.1 Aggiornamento delle Penalità Lagrangiane

Per l'aggiornamento delle penalità si utilizza il metodo del subgradiente come descritto nella sezione (1.4.2).

Si prende il vincolo rilassato e da esso si definisce il subgradiente:

$$\sum_{j \in Lt_i} x_{ij}^k - \sum_{j \in Lr_i} x_{ji}^k - b_i^k = 0 \quad (2.26)$$

$$\downarrow$$

$$s_i^k = \sum_{j \in Lt_i} x_{ij}^k - \sum_{j \in Lr_i} x_{ji}^k - b_i^k \quad (2.27)$$

La variabile s_i^k rappresenta quanto il vincolo rilassato (2.9) viene violato nella soluzione corrente e dipende da k e i poiché il vincolo stesso è definito per ogni combinazione di informazione k e nodo i . Definito il subgradiente, lo si calcola insieme alle variabili x_{ij}^k e si aggiornano le penalità λ_i^k secondo la formula $\lambda_i^k = \lambda_i^k + \theta s_i^k$.

Per scegliere θ si utilizza l'equazione:

$$\theta = \alpha \frac{\text{UpperBound} - \text{LowerBound}}{\|s\|^2} \quad (2.28)$$

I parametri sono scelti in modo da garantire passi ampi in caso di lontananza dall'ottimo, favorendo una convergenza rapida, e passi più piccoli nelle iterazioni vicine all'ottimo, evitando la divergenza dalla soluzione.

- α è un moltiplicatore che viene dimezzato ogni volta che il LowerBound non migliora per un numero prefissato di iterazioni. Se il LowerBound non migliora è probabile si stia lavorando con passi troppo elevati.
- $\text{UpperBound} - \text{LowerBound}$ rappresenta l'ampiezza dell'intervallo in cui si trova la soluzione ottima. Se i due bound sono distanti si consiglia di effettuare passi più grandi mentre al contrario bisogna lavorare con passi più piccoli e una maggiore precisione.
- $\|s\|^2$ è la norma al quadrato del vettore dei subgradienti. Un valore elevato indica una violazione significativa del vincolo e dividendo per esso si riduce il passo, evitando aggiornamenti eccessivi di λ . Al contrario, una norma di valore piccolo indica che si è vicini all'ottimo e quindi dividendo per esso si aumenta il passo evitando una convergenza troppo lenta.

3 Miglioramento della Formulazione Rispetto ai Percorsi Parziali

3.1 Nuova Formulazione

Le formulazioni descritte in precedenza presentano un limite nel fondamento logico: i vincoli (2.2) e (2.9) impongono che ogni Subscriber riceva l'informazione richiesta. Di conseguenza, se anche un solo destinatario non è raggiungibile, tali vincoli risultano violati, il problema non ammette soluzioni e non è possibile perciò calcolare una soluzione parziale, ovvero una in cui solo un sottoinsieme di riceventi viene soddisfatto. Per superare questa limitazione si introduce una nuova variabile decisionale binaria $y_i^k \in \{0, 1\}$ che indica se un determinato nodo i va servito con l'informazione k . Questa variabile ricopre un ruolo chiave in due aspetti del problema: la funzione obiettivo del problema e la definizione di b_i^k .

Per la sua integrazione si aggiorna la seconda formulazione:

$$\text{minimizza } \sum_{k \in K} \sum_{i, j \in A} c_{ij} e_{ij}^k + P \sum_{k \in K} \sum_{t \in \text{NodiTarget}_k} (1 - y_t^k) \quad (3.1)$$

dove P è la penalità per la mancata consegna a un nodo richiedente.

$$b_i^k = \begin{cases} \sum_{t \in \text{NodiTarget}_k} y_t^k & \text{se } i \text{ Publisher} \\ -y_i^k & \text{se } i \text{ Subscriber} \\ 0 & \text{se } i \text{ Broker} \end{cases} \quad (3.2)$$

P è scelto con un valore sufficientemente grande affinché, se un nodo è raggiungibile, risulti sempre più conveniente servirlo piuttosto che pagare la penalità. Idealmente si può pensare a P come il costo complessivo di tutti gli archi della rete sommato a uno. In questo modo, anche un percorso che utilizza tutti gli archi risulta preferibile al mancato servizio del nodo.

$$P = \left(\sum_{i, j \in A} c_{ij} \right) + 1 \quad (3.3)$$

Per decidere quali nodi servire e quali no, si applica lo stesso procedimento basato sul rilassamento lagrangiano già eseguito in precedenza per le variabili x e e . Si riprende dunque il lagrangiano ottenuto al punto 2.5 (equazione (2.21)) e vi si aggiunge il nuovo termine della funzione obiettivo in quanto non altera i calcoli già effettuati:

$$\begin{aligned} \min \sum_{k \in K} \sum_{i, j \in A} (c_{ij} e_{ij}^k + (\lambda_i^k - \lambda_j^k) x_{ij}^k) - \\ \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k + P \sum_{k \in K} \sum_{t \in \text{NodiTarget}_k} (1 - y_t^k) \end{aligned} \quad (3.4)$$

Si isola dunque la seconda parte dell'equazione, contenente la nuova variabile, per ottenere la condizione di attivazione di y :

$$\min P \sum_{k \in K} \sum_{t \in NodiTarget_k} (1 - y_t^k) - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k \quad (3.5)$$

Si separano i nodi nelle diverse tipologie e si sviluppano i prodotti con le corrispondenti λ :

$$\begin{aligned} \min \sum_{k \in K} \sum_{t \in NodiTarget_k} (P - P y_t^k) - \sum_{k \in K} \left(\sum_{br \in Brokers} \lambda_{br}^k b_{br}^k \right. \\ \left. + \sum_{o \in NodoOrigine_k} \lambda_o^k b_o^k + \sum_{t \in NodiTarget_k} \lambda_t^k b_t^k \right) \end{aligned} \quad (3.6)$$

$$\begin{aligned} \downarrow \\ \min \sum_{k \in K} \sum_{t \in NodiTarget_k} (P - P y_t^k) - \sum_{k \in K} \left(\sum_{o \in NodoOrigine_k} \lambda_o^k y_t^k + \sum_{t \in NodiTarget_k} -\lambda_t^k y_t^k \right) \end{aligned} \quad (3.7)$$

(3.6)→(3.7): Nel passaggio tra le due equazioni, si sostituiscono le variabili b con i valori stabiliti dalla definizione.

$$\min \sum_{k \in K} \sum_{t \in NodiTarget_k} (P - P y_t^k) - \sum_{k \in K} \sum_{t \in NodiTarget_k} (y_t^k (\lambda_o^k - \lambda_t^k)) \quad (3.8)$$

(3.8): La sommatoria sui nodi di origine viene rimossa perché, per ogni k , esiste un unico nodo che genera l'informazione.

$$\min \sum_{k \in K} \sum_{t \in NodiTarget_k} (P - P y_t^k) - y_t^k (\lambda_o^k - \lambda_t^k) \quad (3.9)$$

$$\begin{aligned} \downarrow \\ \min \sum_{k \in K} \sum_{t \in NodiTarget_k} P - y_t^k (\lambda_o^k - \lambda_t^k + P) \end{aligned} \quad (3.10)$$

La condizione di attivazione di y_t^k si ottiene confrontando il costo nel caso in cui il nodo target t sia servito ($y_t^k = 1$) rispetto al caso in cui non lo sia ($y_t^k = 0$). Nel momento in cui il costo associato col caso servito risulta minore si ha un guadagno e quindi si attiva y_t^k :

$$P - \lambda_o^k + \lambda_t^k - P < P \rightarrow \lambda_t^k - \lambda_o^k < P \quad (3.11)$$

Pertanto $y_t^k = 1$ quando $\lambda_t^k - \lambda_o^k < P$, con t nodo target e o nodo origine di k .

3.2 Mancata Convergenza del Lower Bound alla Soluzione Intera

La nuova formulazione presenta una criticità sottile ma rilevante: il vincolo (2.11) modificato si configura come un classico esempio di formulazione debole (1.4.3):

$$e_{ij}^k \leq x_{ij}^k \leq e_{ij}^k \sum_{t \in \text{NodiTarget}_k} y_t^k \quad \forall k \in K, (i, j) \in A \quad (3.12)$$

La presenza del fattore $\sum_{t \in \text{NodiTarget}_k} y_t^k$ è la causa diretta di tale debolezza.

3.2.1 Analisi del Problema nel Dettaglio

Il vincolo lega la variabile x_{ij}^k alla variabile binaria di attivazione e_{ij}^k . Seguendo la logica del vincolo ci si aspetterebbe che se non transita alcuna informazione ($x = 0$) allora il vincolo imponga $e = 0$, mentre nel caso contrario costringa $e = 1$. Nella realtà la presenza del fattore $\sum_{t \in \text{NodiTarget}_k} y_t^k$ altera questa dinamica. Prendendo come esempio una rete con un'informazione k che deve raggiungere due target distinti t_1 e t_2 , negli archi in cui la variabile x assume valore singolo ($x_{ij}^k = 1$), il vincolo esprime $1 \leq 2e$, ovvero $e \geq \frac{1}{2}$. Nel rilassamento continuo alla variabile e sarà assegnato il valore minore possibile per minimizzare la funzione obiettivo e quindi assumerà valore 0.5. Nonostante il risolutore costringa la variabile ad assumere valore intero, e quindi 1, il valore frazionario rimarrà impresso nelle λ che guidano l'algoritmo durante le iterazioni successive. L'effetto pratico che si osserva è il seguente: l'algoritmo tende a penalizzare l'arco attivo e a favorire quello disattivo, per poi invertire, mantenendo così una media di attivazione pari a 0.5. Questa oscillazione impedisce al Lower Bound di crescere oltre una certa soglia, poiché blocca il raggiungimento di una soluzione intera ammissibile.

3.2.2 Scrittura in Formulazione Forte

Per eliminare la debolezza del vincolo, si ridefiniscono le variabili x , b e λ in modo da distinguere il nodo target che l'informazione deve raggiungere. Nascono così le variabili x_{ij}^{kt} , b_i^{kt} e λ_i^{kt} .

I nuovi vincoli derivanti sono:

$$x_{ij}^{kt} \leq e_{ij}^k \quad \forall k \in K, t \in \text{NodiTarget}_k, (i, j) \in A \quad (3.13)$$

$$\sum_{j \in Lt_i} x_{ij}^{kt} - \sum_{j \in Lr_i} x_{ji}^{kt} = b_i^{kt} \quad \forall i \in N, t \in \text{NodiTarget}_k, k \in K \quad (3.14)$$

La definizione di b_i^{kt} diventa:

$$b_i^{kt} = \begin{cases} y_i^k & \text{se } i \text{ Publisher dell'informazione } k \text{ diretta al target } t \\ -y_i^k & \text{se } i \text{ Subscriber} \\ 0 & \text{se } i \text{ Broker} \end{cases} \quad (3.15)$$

Con questa formulazione, l'arco viene attivato non appena dell'informazione lo attraversa, per qualsiasi target, ma il legame rimane diretto. In questo modo, se l'arco è disattivato l'informazione passante è nulla e, viceversa, se attivo è limitato a uno. Ciò toglie la possibilità di soluzioni frazionarie e rimuove le oscillazioni descritte in precedenza.

3.2.3 Nuove Condizioni di Attivazione

A seguito della ridefinizione delle variabili decisionali, occorre riformulare le condizioni di scelta per l'attivazione degli archi, l'instradamento delle informazioni e la selezione dei nodi da servire.

Tali condizioni vengono ricalcolate seguendo il medesimo procedimento di rilassamento lagrangiano descritto in precedenza (2.5 e 3.1):

$$\begin{aligned} \min \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + P \sum_{k \in K} \sum_{t \in NodiTarget_k} (1 - y_t^k) + \\ \sum_{k \in K} \sum_{t \in NodiTarget_k} \sum_{i \in N} \lambda_i^{kt} \left(\sum_{j \in Lt_i} x_{ij}^{kt} - \sum_{j \in Lr_i} x_{ji}^{kt} - b_i^{kt} \right) \end{aligned} \quad (3.16)$$

↓

$$\begin{aligned} \min \sum_{k \in K} \left(\sum_{i,j \in A} c_{ij} e_{ij}^k + \sum_{t \in NodiTarget_k} \left(\sum_{i,j \in A} \lambda_i^{kt} x_{ij}^{kt} - \sum_{j,i \in A} \lambda_i^{kt} x_{ji}^{kt} - \right. \right. \\ \left. \left(\sum_{o \in NodoOrigine_{kt}} \lambda_o^{kt} y_t^k + \sum_{ta \in NodiTarget_{kt}} -\lambda_{ta}^{kt} y_t^k \right) \right) + \sum_{t \in NodiTarget_k} P(1 - y_t^k) \end{aligned} \quad (3.17)$$

Essendo unici il nodo di origine o e il nodo target ta , per ogni informazione k e target t , le rispettive sommatorie presentano un unico termine, e possono quindi essere rimosse. Inoltre, poiché ta e t indicano lo stesso nodo, è possibile sostituire ta con t :

$$\begin{aligned} \min \sum_{k \in K} \left(\sum_{i,j \in A} c_{ij} e_{ij}^k + \sum_{t \in NodiTarget_k} \sum_{i,j \in A} (\lambda_i^{kt} - \lambda_j^{kt}) x_{ij}^{kt} + \right. \\ \left. \sum_{t \in NodiTarget_k} (P - P y_t^k) - \sum_{t \in NodiTarget_k} (\lambda_o^{kt} - \lambda_t^{kt}) y_t^k \right) \end{aligned} \quad (3.18)$$

La funzione obiettivo può essere suddivisa in due sotto-problemi indipendenti, ciascuno utilizzato per ricavare le condizioni di attivazione delle relative variabili decisionali al loro interno.

Il primo sotto-problema permette di ricavare le condizioni per e_{ij}^k e x_{ij}^{kt} :

$$\min \sum_{k \in K} \sum_{i,j \in A} (c_{ij} e_{ij}^k + \sum_{t \in NodiTarget} (\lambda_i^{kt} - \lambda_j^{kt}) x_{ij}^{kt}) \quad (3.19)$$

Al fine di attivare e_{ij}^k si confronta il caso attivo con quello disattivo, mirando a una riduzione del valore della funzione obiettivo:

$$c_{ij} + \sum_{t \in NodiTarget} (\lambda_i^{kt} - \lambda_j^{kt}) x_{ij}^{kt} < 0 \quad (3.20)$$

Nello specifico, se $(\lambda_i^{kt} - \lambda_j^{kt}) < 0$, si pone $x_{ij}^{kt} = 1$ altrimenti $x_{ij}^{kt} = 0$.

Il secondo sotto-problema permette di ricavare la condizione per y_t^k :

$$\min \sum_{k \in K} (\sum_{t \in NodiTarget_k} (P - P y_t^k) - \sum_{t \in NodiTarget} (\lambda_o^{kt} - \lambda_t^{kt}) y_t^k) \quad (3.21)$$

Lo sviluppo dell'equazione porta alla condizione descritta in precedenza (3.11), rimasta invariata: $\lambda_t^{kt} - \lambda_o^{kt} < P$.

3.2.4 Terza Formulazione Completa

Funzione Obiettivo:

$$\text{minimizza } \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + P \sum_{k \in K} \sum_{t \in NodiTarget_k} (1 - y_t^k) \quad (3.22)$$

Vincoli:

$$\sum_{j \in Lt_i} x_{ij}^{kt} - \sum_{j \in Lr_i} x_{ji}^{kt} = b_i^{kt} \quad \forall i \in N, t \in NodiTarget_k, k \in K \quad (3.23)$$

$$\sum_{k \in K} a^k e_{ij}^k \leq u_{ij} \quad \forall (i, j) \in A \quad (3.24)$$

$$x_{ij}^{kt} \leq e_{ij}^k \quad \forall k \in K, t \in NodiTarget_k, (i, j) \in A \quad (3.25)$$

$$x_{ij}^{kt} \geq 0 \quad \forall (i, j) \in A, k \in K \quad (3.26)$$

$$e_{ij}^k \in \{0, 1\} \quad \forall (i, j) \in A, k \in K \quad (3.27)$$

$$y_t^k \in \{0, 1\} \quad \forall k \in K, t \in NodiTarget_k \quad (3.28)$$

4 Implementazione Algoritmica

In questo capitolo vengono presentati e descritti gli algoritmi sviluppati, illustrandone la logica di funzionamento strutturata su due fasi. Nella prima fase si calcola un Upper Bound mediante metodi euristici, con l'obiettivo di contenere i tempi di calcolo che, altrimenti, risulterebbero proibitivi. Nella seconda fase, tale valore viene utilizzato come riferimento per guidare la ricerca del miglior Lower Bound, sfruttando l'aggiornamento delle penalità lagrangiane descritto in precedenza (2.5.1). Il processo, infine, si conclude con un confronto volto a valutare la qualità della soluzione ottenuta e la sua distanza dall'ottimo.

Le principali classi di algoritmi implementate sono:

- **Algoritmo Cplex:** Il modello matematico viene direttamente implementato nell'ambiente di sviluppo di Cplex o in python. La soluzione ottima restituita costituisce il riferimento per il confronto con i risultati degli altri algoritmi.
- **Algoritmo Lagrangiano:** Algoritmo implementato in Python, applica la logica teorica del rilassamento lagrangiano e delle condizioni di attivazione per il calcolo del Lower Bound ottimo.
- **Algoritmo euristico per l'Upper Bound:** Algoritmo euristico ispirato alla logica di Dijkstra, che viene utilizzato per la ricerca rapida di un Upper Bound ammissibile.

Ogni algoritmo presentato in questo capitolo utilizza le variabili già definite nel paragrafo dei simboli globali (2.2).

4.1 Implementazione delle Formulazioni Senza Soluzioni Parziali

4.1.1 Upper Bound Euristico

Per il calcolo dell'Upper Bound relativo alla prima e alla seconda formulazione, si è optato per lo sviluppo di un algoritmo euristico basato sulla logica di Dijkstra per i cammini minimi. Poiché l'ordine di instradamento delle informazioni influenza significativamente l'allocation delle capacità sugli archi e la qualità della soluzione finale, l'algoritmo adotta un approccio Multi-Start ripetuto per un numero arbitrario di iterazioni. Ad ogni iterazione, l'insieme delle informazioni viene mescolato casualmente e per ciascuna di esse viene eseguita una variante dell'algoritmo di Dijkstra. Questo integra un controllo preventivo sulle capacità residue degli archi: un nodo adiacente viene preso in considerazione solo se l'arco che lo raggiunge è in grado di sostenere il peso dell'informazione corrente. Al termine di ogni iterazione, i cammini ottimi vengono ricostruiti a ritroso a partire dai nodi target, aggiornando dinamicamente le capacità residue della rete. L'algoritmo, infine, restituisce i percorsi che hanno il costo totale complessivo minimo.

Le variabili in input coincidono con quelle definite nel paragrafo (2.2).

Nella prima parte di codice si definiscono le strutture globali utilizzate per memorizzare il cammino ottimo e il corrispondente costo.

```
percorsoOttimo=[]  
costoTot = np.inf
```

All'inizio di ogni iterazione dell'algoritmo viene creata una copia delle strutture necessarie per i calcoli oltre al mescolamento dell'ordine di elaborazione delle informazioni.

```
costo = 0.0  
strade = A.copy()  
CapacitàUsata = u.copy()  
informazioni = K.copy()  
random.shuffle(informazioni)  
percorsi = []  
soluzioneValida = True
```

L'algoritmo procede iterando su ogni informazione e ad ogni iterazione salva l'insieme di nodi da visitare filtrati per raggiungibilità.

```
if not soluzioneValida:  
    break  
unvisitedNodes = [i for i in N]  
for g,l in Lr.items():  
    if len(l)==0:  
        unvisitedNodes.remove(g)  
nodoDiPartenza = NodoOrigine[informazione]  
if nodoDiPartenza not in unvisitedNodes:  
    unvisitedNodes.append(nodoDiPartenza)
```

Successivamente viene applicata una variante dell'algoritmo di Dijkstra con controllo sulle capacità per determinare l'albero dei cammini minimi. Per l'elaborazione viene utilizzata una tabella in grado di memorizzare il costo del cammino minimo temporaneo e il relativo nodo predecessore. In caso di nodi non raggiungibili l'algoritmo viene fermato forzatamente.

```
while (len(unvisitedNodes)>0):  
    for (i,j) in strade:  
        if(i==nodoProtagonista and j in unvisitedNodes):  
            if CapacitàUsata[(i, j)] - a[informazione] >= 0:  
                if (tabella[j][1]>tabella[i][1]+C[i, j]):  
                    tabella[j][1]=tabella[i][1]+C[i, j]  
                    tabella[j][2]=nodoProtagonista  
            unvisitedNodes.remove(nodoProtagonista)  
            indiceProssimoNodo = -1  
    for j in unvisitedNodes:  
        if tabella[j][1]<math.inf:  
            if indiceProssimoNodo == -1:  
                indiceProssimoNodo = j
```

```

        else :
            if ( tabella [ j ][ 1 ] < tabella [ indiceProssimoNodo ][ 1 ] ):
                indiceProssimoNodo = j
    if indiceProssimoNodo == -1:
        unvisitedNodes = []
    else :
        nodoProtagonista = indiceProssimoNodo

```

Completata l'esplorazione della rete, i percorsi ottimi vengono ricostruiti a ritroso a partire dai rispettivi nodi target e viene aggiornata la capacità rimanente della rete.

```

nodiFinali = NodiTarget[informazione].copy()
random.shuffle(nodiFinali)
for nodoFinale in nodiFinali:
    percorso = []
    percorso.append(nodoFinale)
    if ( tabella [ nodoFinale ][ 2 ] == None ):
        costo = np.inf
        soluzioneValida = False
        break
    while ( tabella [ nodoFinale ][ 2 ] != None ):
        preFinale = tabella [ nodoFinale ][ 2 ]
        percorso.append(preFinale)
        CapacitàUsata[(preFinale, nodoFinale)] = CapacitàUsata[(preFinale, nodoFinale)] - a[informazione]
        costo += C[preFinale, nodoFinale]
        nodoFinale = preFinale
    percorsi.append(informazione)
    percorsi.append(percorso)

```

Al termine del ciclo di controllo su tutte le informazioni, se la soluzione è ammissibile viene confrontata con la precedente migliore e se la precedente risulta peggiore in termini di costo viene sostituita.

```

if soluzioneValida and costo < costoTot:
    costoTot = costo
    percorsoOttimo = percorsi

```

4.1.2 Prima Formulazione

Per quanto riguarda la prima formulazione, lo sviluppo si è concentrato sulla creazione del modello in ambiente di sviluppo Cplex e in python, escludendo l'implementazione della metodologia basata sul rilassamento lagrangiano.

Cplex

Nell'ambiente Cplex il codice risulta molto lineare, con una distinzione chiara tra la funzione obiettivo e i vincoli.

```
minimize
sum(k in K, i in N, j in Lr[i]) C[<j,i>] * x[k][<j,i>];
subject to {
    forall(k in K, t in NodiTarget[k])
        sum(j in Lr[t]) x[k][<j,t>] == 1;

    forall(k in K, i in Brokers)
        sum(j in Lr[i]) x[k][<j,i>] == sum(j in Lt[i]) x[k][<i,j>];

    forall(k in K, <i,j> in A)
        bigM*e[k][<i,j>] >= x[k][<i,j>];

    forall(i in N, j in Lt[i])
        sum(k in K) a[k]*(e[k][<i,j>]+e[k][<j,i>])<=u[<i,j>];
}
```

Python API

Nel codice in python la scrittura si complica e le definizioni si fanno più complesse. Inizialmente vengono definite e generate le variabili decisionali con i rispettivi vincoli associati: (2.6) e (2.7).

```
modello = cplex.Cplex()
modello.objective.set_sense(modello.objective.sense.minimize)
x = [(k,i,j) for k in K for (i,j) in A]
modello.variables.add(names=[f"x_{k}_{i}_{j}" for (k,i,j) in x],
lb=[0]*len(x),
types=["I"]*len(x))
e = [(k,i,j) for k in K for (i,j) in A]
modello.variables.add(names=[f"e_{k}_{i}_{j}" for (k,i,j) in e],
lb=[0]*len(e),
ub=[1]*len(e),
types=["B"]*len(e))
```

Generate le variabili funzionali si procede con la definizione della funzione obiettivo e dei vincoli del problema.

Funzione Obiettivo (2.1):

```
temp_X = []
temp_Coeffs=[]
for k in K:
    for i in N:
        for j in Lr[i]:
            if(j,i) in A:
```

```

        temp_Coeffs.append(C[j, i])
        temp_X.append(f"x_{k}_{j}_{i}")
    modello.objective.set_linear(list(zip(temp_X, temp_Coeffs)))

```

Primo vincolo (2.2):

```

for k in K:
    for t in NodiTarget[k]:
        temp_X = []
        temp_Coeffs = []
        for j in Lr[t]:
            if (j, t) in A:
                temp_X.append(f"x_{k}_{j}_{t}")
                temp_Coeffs.append(1)
        modello.linear_constraints.add
        (lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["E"], rhs=[1])

```

Secondo vincolo (2.3):

```

for k in K:
    for i in Brokers:
        temp_X = []
        temp_Coeffs = []
        for j in Lr[i]:
            if (j, i) in A:
                temp_X.append(f"x_{k}_{j}_{i}")
                temp_Coeffs.append(1)
        for j in Lt[i]:
            if (i, j) in A:
                temp_X.append(f"x_{k}_{i}_{j}")
                temp_Coeffs.append(-1)
        modello.linear_constraints.add
        (lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["E"], rhs=[0])

```

Terzo vincolo (2.4):

```

for k in K:
    for (i, j) in A:
        temp_X = []
        temp_Coeffs = []
        temp_X.append(f"e_{k}_{i}_{j}")
        temp_Coeffs.append(bigM)
        temp_X.append(f"x_{k}_{i}_{j}")
        temp_Coeffs.append(-1)
        modello.linear_constraints.add
        (lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["G"], rhs=[0])

```

Quarto vincolo (2.5):

```

for i in N:

```

```

for j in Lt[i]:
    temp_X=[]
    temp_Coeffs=[]
    for k in K:
        if(i,j) in A:
            temp_X.append(f"e_{k}_{i}_{j}")
            temp_Coeffs.append(a[k])
        if(j,i) in A:
            temp_X.append(f"e_{k}_{j}_{i}")
            temp_Coeffs.append(a[k])
    modello.linear_constraints.add
    (lin_expr=[cplex.SparsePair(temp_X,temp_Coeffs)], senses=["L"], rhs=[u[i,j]])

```

4.1.3 Seconda Formulazione

La seconda formulazione, oltre all'implementazione nell'ambiente Cplex e in python tramite API, prevede lo sviluppo di un algoritmo basato sulla teoria del rilassamento lagrangiano, strutturato in diverse funzioni, ognuna con il proprio specifico ruolo.

Calcolo Costi Penalizzati

Il calcolo dei costi penalizzati si basa sull'equazione (2.25), la quale quantifica il guadagno associato all'attivazione di un determinato arco. Qualora tale operazione determini una riduzione del valore della funzione obiettivo, l'arco viene inserito all'interno di una lista di candidati per una futura elaborazione.

```

for (k,i,j) in d:
    if (lam[(k,i)]-lam[(k,j)])<0:
        d[(k,i,j)]=C[i,j]+((lam[(k,i)]-lam[(k,j)])*len(NodiTarget[k]))
    else:
        d[(k,i,j)] = C[i,j]
    if (d[(k,i,j)]<0):
        ePerKnapsack.append((k,i,j))

```

KnapSack

Al fine di garantire il rispetto del vincolo sulla capacità degli archi, si è optato per lo sviluppo di un algoritmo basato sulla programmazione dinamica applicata al problema dello zaino (KnapSack problem). Il problema dello zaino è un problema molto famoso nel panorama della ricerca operativa, il cui scopo è quello di inserire la migliore combinazione di oggetti possibile all'interno di uno zaino dotato di una capacità massima. Ogni oggetto è caratterizzato da un peso e da un guadagno, di conseguenza l'obiettivo finale è la massimizzazione del profitto totale. Trattandosi

di un problema complesso, i tempi di risoluzione risultano molto elevati con metodi tradizionali. Per superare questo limite si sfrutta la programmazione dinamica ovvero si suddivide in sotto-problemi e per il calcolo di ciascuno di essi si sfruttano i calcoli già effettuati in quelli precedenti al fine di ridurre i tempi di calcolo. La capacità massima dello zaino coincide con la capacità dell'arco, il peso degli oggetti da inserire all'interno dello zaino con la banda occupata dall'informazione e il profitto è rappresentato dal valore opposto del costo penalizzato, che viene reso positivo per consentirne l'utilizzo diretto. In caso di attivazione dell'arco la relativa variabile x verrà aggiornata seguendo la logica espressa nel paragrafo (2.5). In primo luogo, a ciascun arco vengono associate le informazioni che si ritengono candidate a transitarvi.

```
kPerArco = {}
for (k,i,j) in ePerKnapsack:
    if (i,j) not in kPerArco:
        kPerArco[(i,j)] = []
        kPerArco[(i,j)].append(k)
```

Successivamente si esegue l'algoritmo dello zaino per ogni arco, traducendo i parametri nelle apposite strutture dati.

```
for (i,j) in kPerArco.keys():
    valoriD = []
    peso = u[(i,j)]
    pesi = []
    valoriD.append(0)
    pesi.append(0)
    listaK = []
    listaK.append(None)
    for k in kPerArco[(i,j)]:
        valoriD.append(-l*d[(k,i,j)])
        pesi.append(a[k])
        listaK.append(k)
```

Si procede quindi alla generazione della matrice di programmazione dinamica all'interno della quale vengono memorizzati i valori ottimi dei relativi sotto-problemi, divisi in base alla capacità dello zaino e agli oggetti selezionabili.

```
tabella = []
for z in range(len(valoriD)):
    riga = []
    for y in range(peso+1):
        riga.append(0)
    tabella.append(riga)
for z in range(1,len(valoriD)):
    for y in range(1,peso+1):
        if (y>=pesi[z]):
            tabella[z][y]=max(tabella[z-1][y], tabella[z-1][y-pesi[z]]+valoriD[z])
        else:
```

```
tabella[z][y]=tabella[z-1][y]
```

Finito il riempimento della tabella, la soluzione ottima è identificata dalla cella basso a destra nella matrice. Partendo da esso si esegue un controllo a ritroso per identificare le informazioni scelte da far passare e si aggiorna la e e la x .

```
migliore = tabella[len(valoriD)-1][peso]
rigaDelMigliore = len(valoriD)-1
colonnaDelMigliore = peso
while (rigaDelMigliore > 0):
    if (migliore != tabella[rigaDelMigliore-1][colonnaDelMigliore]):
        k = listaK[rigaDelMigliore]
        e[(k,i,j)] = 1.0
        if (lam[k,i]-lam[k,j] >= 0):
            x[(k,i,j)] = 1
        else:
            x[(k,i,j)] = len(NodiTarget[k])
            colonnaDelMigliore = colonnaDelMigliore - pesi[rigaDelMigliore]
            rigaDelMigliore = rigaDelMigliore - 1
            migliore = tabella[rigaDelMigliore][colonnaDelMigliore]
    else:
        rigaDelMigliore = rigaDelMigliore - 1
```

Calcola Step

Per determinare l'importanza dell'aggiornamento da applicare alle penalità lagrangiane, viene calcolato lo step del subgradiente con la formula descritta (2.28).

```
listaSubGradienti = []
for k in K:
    for i in N:
        listaSubGradienti.append(subGradient[(k,i)])
norma = np.linalg.norm(listaSubGradienti)
norma = norma.item()
if norma != 0:
    valoreDiCambio = alpha * ((Ub - lastLb[-1]) / (norma ** 2))
else:
    valoreDiCambio = alpha * ((Ub - lastLb[-1]))
```

Calcolo del Lower Bound attuale

Con l'obiettivo di monitorare la convergenza dell'algoritmo e di aggiornare lo step, il valore del Lower Bound viene ricalcolato ad ogni iterazione eseguendo il calcolo della funzione obiettivo lagrangiana (2.20).

```
Lb=0.0
for (i,j) in A:
```

```

    for k in K:
        Lb += d[(k,i,j)]*e[(k,i,j)]
for k in K:
    for i in N:
        Lb -= lam[(k,i)]*b[k][i]

```

Aggiornamento Penalità Lagrangiane

Sfruttando lo step precedentemente calcolato, si procede all'aggiornamento delle penalità lagrangiane.

```

subGradianti = {(k,i):0.0 for k in K for i in N}
for k in K:
    for i in N:
        Sommatoria1 = 0.0
        Sommatoria2 = 0.0
        for j in Lt[i]:
            Sommatoria1 += x[(k,i,j)]
        for j in Lr[i]:
            Sommatoria2 += x[(k,j,i)]
        subGradianti[(k,i)] = Sommatoria1 - Sommatoria2 - b[k][i]
valoreDiCambio = calcolaValoreDiCambio(lastLb, subGradianti, Ub, alpha)
for k in K:
    for i in N:
        lam[(k,i)] = lam[(k,i)] + valoreDiCambio * subGradianti[(k,i)]

```

Main Loop

Infine, le funzioni appena elencate vengono gestite all'interno di un ciclo principale, il quale itera i metodi fino al raggiungimento del Lower Bound ottimo o all'esecuzione di un numero arbitrario di cicli. In questa implementazione il parametro alpha per la riduzione dello step viene dimezzato se non si incontra un miglioramento del Lower Bound per 20 cicli di fila.

```

lam = {(k,i):0.0 for k in K for i in N}
d = {(k,i,j):0 for k in K for (i,j) in A}
e = {(k,i,j):0 for k in K for (i,j) in A}
x = {(k,i,j):0.0 for k in K for (i,j) in A}
ePerKnapsack = []
ultimiLowerBounds = [0]
Lb=0.0
alpha = 0.1
it_max=10000
alphaterazioni=0
iterazioni=0
Ub,percorso = DijkstraEuristico()
while abs(Ub-Lb)>1e-4 and iterazioni<it_max:

```

```

iterazioni+=1
ePerKnapsack=[]
e = {(k,i,j):0 for k in K for (i,j) in A}
x = {(k,i,j):0 for k in K for (i,j) in A}
CalcolaCostiPenalizzati(lam,d,ePerKnapsack)
KnapSack(ePerKnapsack,d,e,x)
Lb = calcolaMin(lam)
ultimiLowerBounds.append(Lb)
if(ultimiLowerBounds[len(ultimiLowerBounds)-1]<=ultimiLowerBounds[len(
ultimiLowerBounds)-2]):
    alphaterazioni+=1
if(alphaterazioni>20):
    alphaterazioni=0
    alpha=alpha/2
else:
    alphaterazioni=0
aggiornaLambda(lam,ultimiLowerBounds,Ub,alpha)

```

Cplex

Similmente alla formulazione precedente, l'ambiente Cplex consente una scrittura del codice lineare. In particolare, poiché il vincolo (2.9) incorpora due vincoli distinti (2.2) e (2.3), il modello finale presenta un vincolo in meno rispetto alla prima formulazione.

```

minimize
sum(k in K, <i,j> in A) C[<i,j>] * e[k][<i,j>];
subject to {
    forall(k in K, i in N : i != NodoOrigine[k])
        sum(j in Lt[i]) x[k][<i,j>] - sum(j in Lr[i]) x[k][<j,i>] == b[k][i];

    forall(<i,j> in A)
        sum(k in K) a[k] * e[k][<i,j>] <= u[<i,j>];

    forall(k in K, <i,j> in A) {
        e[k][<i,j>] <= x[k][<i,j>];
        x[k][<i,j>] <= card(NodiTarget[k]) * e[k][<i,j>];
    }
}

```

Python API

Per quanto riguarda l'implementazione in python tramite le API di cplex, il terzo vincolo deve essere suddiviso in due sotto-vincoli per garantire il rispetto di entrambe le condizioni.

Inizialmente, vengono inizializzate le variabili decisionali con i relativi vincoli associati (2.12) e (2.13).

```

modello = cplex.Cplex()
modello.objective.set_sense(modello.objective.sense.minimize)
x = [(k,i,j) for k in K for (i,j) in A]
modello.variables.add(names=[f"x_{k}_{i}_{j}" for (k,i,j) in x],
lb=[0]*len(x),
types=["I"]*len(x))
e = [(k,i,j) for k in K for (i,j) in A]
modello.variables.add(names=[f"e_{k}_{i}_{j}" for (k,i,j) in e],
lb=[0]*len(e),
ub=[1]*len(e),
types=["B"]*len(e))

```

Istanziare le variabili, si procede con la definizione della funzione obiettivo e dei vincoli della formulazione. Funzione Obiettivo (2.8):

```

temp_X = []
temp_Coeffs=[]
for k in K:
    for (i,j) in A:
        temp_Coeffs.append(C[i,j])
        temp_X.append(f"e_{k}_{i}_{j}")
modello.objective.set_linear(list(zip(temp_X, temp_Coeffs)))

```

Primo vincolo (2.9):

```

for k in K:
    for i in N:
        if i != NodoOrigine[k]:
            temp_X = []
            temp_Coeffs=[]
            for j in Lt[i]:
                if (i,j) in A:
                    temp_X.append(f"x_{k}_{i}_{j}")
                    temp_Coeffs.append(1)
            for j in Lr[i]:
                if (j,i) in A:
                    temp_X.append(f"x_{k}_{j}_{i}")
                    temp_Coeffs.append(-1)
            modello.linear_constraints.add
            (lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)],
senses=["E"], rhs=[b[k][i]])

```

Secondo vincolo (2.10)

```

for (i,j) in A:
    temp_X = []
    temp_Coeffs=[]
    for k in K:
        temp_X.append(f"e_{k}_{i}_{j}")

```

```

temp_Coeffs.append(a[k])
modello.linear_constraints.add
(lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["L"], rhs=[u[i, j]])

```

Terzo vincolo (2.11)

```

for k in K:
    for (i, j) in A:
        temp_X=[]
        temp_Coeffs=[]
        temp_X.append(f"e_{k}_{i}_{j}")
        temp_Coeffs.append(1)
        temp_X.append(f"x_{k}_{i}_{j}")
        temp_Coeffs.append(-1)
        modello.linear_constraints.add
        (lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["L"], rhs=[0])

for k in K:
    for (i, j) in A:
        temp_X=[]
        temp_Coeffs=[]
        temp_X.append(f"x_{k}_{i}_{j}")
        temp_Coeffs.append(1)
        temp_X.append(f"e_{k}_{i}_{j}")
        temp_Coeffs.append(-len(NodiTarget[k]))
        modello.linear_constraints.add
        (lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["L"], rhs=[0])

```

4.2 Formulazione Con Soluzioni Parziali

4.2.1 Dijkstra a Scenari e Meccanismo di Discesa dell'Upper Bound

Per quando riguarda la nuova formulazione, l'approccio alla ricerca di un Upper Bound ammissibile è stato radicalmente cambiato al fine di superare i limiti della precedente versione. Nel modello antecedente dell'algoritmo, l'impossibilità di raggiungere anche un solo nodo target da parte della rispettiva informazioni decretava il fallimento dell'intera esecuzione. Nella nuova versione, divisa in funzioni, il calcolo di un Upper Bound euristico è composto da una base logica che segue la procedura di Dijkstra per i cammini minimi unita con un controllo sulla capacità residua degli archi della rete. La definizione di euristica deriva dalla valutazione, da parte dell'algoritmo, di un sottoinsieme di permutazioni scelte casualmente sia delle informazioni, sia dei target da raggiungere. Le due principali innovazioni introdotte in questa versione consistono nella gestione di scenari multipli di esplo-

razione delle soluzioni e del meccanismo di stringimento dell'Upper Bound.

La prima novità riguarda la strutturazione di soluzioni seguendo scenari alternativi per la scelta del percorso. Qualora l'algoritmo incontri più strade per il raggiungimento di un nodo target di pari costo, non sceglie casualmente quale memorizzare e utilizzare ma, al contrario, crea uno scenario distinto per ogni strada ottima individuata. Con questa modalità la ricerca del percorso ottimo per le informazioni e target successivi non risente di una decisione casuale, ma viene testata in parallelo insieme agli altri scenari possibili. Al termine del processo vengono estrapolati gli scenari migliori individuati e i relativi costi associati. Al fine di rimanere in linea con l'ideologia di un algoritmo rapido e euristico viene imposto un limite massimo sia alle soluzioni finali sia agli scenari da mantenere attivi durante l'esecuzione.

La seconda novità risiede nella convergenza dell'Upper Bound verso la soluzione ottima. Poiché la determinazione dell'Upper Bound migliore risulta particolarmente dispendiosa nella prima iterazione sebbene si utilizzi una procedura euristica, si adotta una strategia di aggiornamento ripetuto. A intervalli prefissati di iterazioni, l'Upper Bound viene ricalcolato utilizzando il medesimo codice ma aggiungendo una penalizzazione ai costi degli archi sfruttando le penalità lagrangiane.

Algoritmo di Dijkstra con Controllo Capacità

Questa versione dell'algoritmo è stata ottimizzata utilizzando la libreria `heapq` di python, la quale implementa una struttura dati con priorità per semplificare il codice.

```
tabella={}
nodiDaVisitare=[]
for nodo in N:
    if nodo == sorgente:
        tabella[sorgente]=[0,[]]
    else:
        tabella[nodo]=[math.inf,[]]
heapq.heappush(nodiDaVisitare,(0,sorgente))
nodiVisitati=[]
while len(nodiDaVisitare)>0:
    costo,nodo = heapq.heappop(nodiDaVisitare)
    nodiVisitati.append(nodo)
    archi = [(i,j) for i,j in A if i == nodo]
    for i,j in archi:
        if capacità[(i,j)]>=banda:
            if tabella[j][0] > tabella[i][0]+costi[(i,j)]:
                tabella[j][0] = tabella[i][0]+costi[(i,j)]
                tabella[j][1]=[]
                tabella[j][1].append(i)
    if(j not in nodiVisitati):
```

```

        heapq.heappush(nodiDaVisitare,(tabella[j][0],j))
    else:
        if tabella[j][0] == tabella[i][0]+costi[(i,j)]:
            tabella[j][1].append(nodo)
if tabella[target][0]==math.inf:
    return []
costo = tabella[target][0]
percorsi = ricostruisciPercorsi(target,sorgente,tabella,k)
return [(costo,percorso) for percorso in percorsi]

```

La chiamata alla funzione di Dijkstra modificata viene gestita da un metodo centrale che richiede in ingresso una permutazione delle informazioni. Per ciascuna di esse, la funzione itera su un numero prefissato di sottoinsiemi di target da raggiungere, mescolati casualmente, gestendo in contemporanea i diversi scenari attivi. Nello specifico, all'interno del metodo è stato inserito un meccanismo di controllo per garantire il passaggio gratuito attraverso un arco se esso, per la stessa informazione, è già stato attivato nello scenario corrente.

Per la creazione si procede alla copia della capacità originale della rete e alla definizione della struttura che conterrà i diversi scenari.

La struttura è pensata in modo da avere per ogni soluzione:

```

capacità = u.copy()
soluzioni = [[0,[],capacità,[],[]]]

```

La struttura è pianificata in modo da contenere per ogni scenario:

- Costo complessivo
- Archi attivi
- Capacità della rete residua
- Target raggiunti
- Target irraggiungibili

Successivamente, l'algoritmo itera su ogni informazione k e su ciascuna permutazione dei nodi target per la relativa informazione. Durante l'iterazione, il metodo considera tutti gli scenari precedentemente memorizzati e, per ogni target, cerca di espanderli. Se il cammino minimo verso il target è univoco, lo scenario corrente viene aggiornato e si prosegue altrimenti, si crea un numero di scenari pari alle alternative ottime individuate. Qualora invece il target risulti non raggiungibile, il nodo viene catalogato come inarrivabile all'interno dello scenario.

```

for k in permutazione:
    soluzioniPerPermutazione = []
    for permutazioneTarget in permutazioniTarget:
        for soluzione in soluzioni:
            soluzioniTemp=[]

```

```

soluzioniTemp.append(soluzione.copy())
for target in permutazioneTarget:
    nuoviTemp=[]
    for soluzioneTemp in soluzioniTemp:
        valori = copy.deepcopy(soluzioneTemp)
        costiConsiderandoArchiAttivi = copy.deepcopy(C)
        capacitàConsiderandoArchiAttivi = copy.deepcopy(soluzioneTemp[2])
        archiAttivi = []
        for kArco,i,j in soluzioneTemp[1]:
            if k==kArco:
                costiConsiderandoArchiAttivi[(i,j)]=0
                capacitàConsiderandoArchiAttivi[(i,j)] += a[k]
                archiAttivi.append((i,j))
        percorsi = Dijkstra(NodoOrigine[k],target,
        capacitàConsiderandoArchiAttivi,a[k],k,
        costiConsiderandoArchiAttivi)
        if len(percorsi)==0:
            copia = copy.deepcopy(valori)
            copia[4].append((k,target))
            nuoviTemp.append
            ([copia[0],copia[1],copia[2],copia[3],copia[4]])
        else:
            for i in range(len(percorsi)):
                costo,archi = percorsi[i]
                cap = nuovaCapacità(soluzioneTemp[2],archi,a[k],
                archiAttivi)
                copia=copy.deepcopy(valori)
                copia[1].extend(archi)
                copia[3].append((k,target))
                nuoviTemp.append([copia[0]+costo,copia[1],cap,copia[3],
                copia[4]])
            soluzioniTemp=nuoviTemp
        soluzioniPerPermutazione.extend(soluzioniTemp)
    soluzioniPerPermutazione.sort(key=lambda s: (-len(s[3]), s[0]))
    soluzioni=soluzioniPerPermutazione[:soluzioniDaContinuare]
return soluzioni

```

Per l'aggiornamento della capacità degli archi nella rete viene sviluppato una semplice funzione di riduzione dei valori.

```

cap = capacità.copy()
for arco in archi:
    if arco not in archiAttivi:
        arco = arco[1:3]
        cap[arco] -= banda

```

Per la ricostruzione del percorso si è optato per l'utilizzo di una funzione generatrice ricorsiva. Il metodo risale la rete a ritroso restituendo mediante l'istruzione yield tutti i cammini minimi alternativi individuati.

```

if nodo == sorgente:
    yield []
    return
for predecessore in tabella[nodo][1]:
    for percorso in ricostruisciPercorsi(predecessore , sorgente , tabella ,k):
        yield percorso + [(k,predecessore ,nodo)]

```

Infine, la gestione delle permutazione delle informazioni, la valutazione delle soluzioni e il corretto calcolo del costo della soluzione in caso di target non raggiunti sono coordinate una funzione che funge come punto di partenza dell'intero algoritmo:

```

for permutazione in permutazioniInformazioni:
    soluzioni = MainEuristico(permutazione)
    soluzioniTotali.extend(soluzioni)
    if len(soluzioniTotali) > soluzioniMax:
        break
if len(soluzioniTotali)==0:
    return
maxRaggiunti = max(len(soluzione[3]) for soluzione in soluzioniTotali)
candidati = [soluzione for soluzione in soluzioniTotali if
len(soluzione[3])==maxRaggiunti]
costoMin = min(soluzione[0] for soluzione in candidati)
migliori = [soluzione for soluzione in candidati if soluzione[0]==costoMin]
costoMinModificato = costoMin+P*len(migliori[0][4])

```

Calcolo dei Costi con Penalità Lagrangiane

Per la convergenza dell'Upper Bound, il metodo centrale viene modificato per integrare nei pesi degli archi le penalità lagrangiane, calcolate dal metodo del rilassamento. È importante evidenziare che i costi utilizzati durante la fase di instradamento non sono più validi per il calcolo del costo complessivo della rete, poiché incrementati dalle λ . Risulta perciò necessario ricalcolarli per determinare il costo della soluzione.

```

for soluzioneTemp in soluzioniTemp:
    costiLambda = {}
    for i,j in A:
        costiLambda[(i,j)] = C[(i,j)]+max(0,lam[(k,target,i)]-lam[(k,target,j)])
    for kArco, i, j in soluzioneTemp[1]:
        if k == kArco:
            costiLambda[(i,j)] = 0
        percorsi = Dijkstra(NodoOrigine[k],target ,capacitàConsiderandoArchiAttivi ,
a[k],k, costiLambda)
    if len(percorsi)==0:
        ...
    else:

```

```

for i in range(len(percorsi)):
    costo, archi = percorsi[i]
    costo=0
    for kArco, nodoI, nodoJ in archi:
        if (nodoI,nodoJ) not in archiAttivi:
            costo += C[(nodoI,nodoJ)]
    nuoviTemp.append([copia[0]+costo, copia[1], cap, copia[3], copia[4]])

```

4.2.2 Terza formulazione

Lagrangiano

L'implementazione della terza formulazione usa come base le funzioni già sviluppate per la seconda, adattandole per consentire la gestione sia della variabile decisionale nuova sia di quelle vecchie modificate, in accordo con la teoria (3.2.2).

Calcola Y

A tale scopo, è stata definita una nuova funzione usata per determinare l'attivazione della variabile y , seguendo la condizione estrapolata dal rilassamento lagrangiano (3.2.3).

```

for k in K:
    for t in NodiTarget[k]:
        if lam[(k, t, t)] - lam[(k, t, NodoOrigine[k])] < P:
            y[(k, t)] = 1
        else:
            y[(k, t)] = 0

```

Calcola B

Per rispettare la definizione teorica della variabile b (3.15), si implementa un metodo di aggiornamento che opera in funzione dei valori assunti da y . Tale metodo viene richiamato a ogni iterazione, garantendo la coerenza tra le due variabili durante l'esecuzione dell'algoritmo.

```

b = {k: {t: {i: 0 for i in N} for t in NodiTarget[k]} for k in K}
for k in K:
    for t in NodiTarget[k]:
        b[k][t][NodoOrigine[k]] = y[(k, t)]
        b[k][t][t] = -y[(k, t)]

```

Calcolo Costi Penalizzati

La funzione per il calcolo dei costi penalizzati viene modificata in modo da restare conforme con l'aggiornamento della formula decisionale per l'attivazione degli archi (3.2.3). Quest'ultima, infatti, in aggiunta alla versione precedente, richiede un controllo di tutti i nodi target che si intendono raggiungere.

```
for (k,i,j) in d:
    costo = C[i,j]
    for t in NodiTarget[k]:
        if (lam[(k,t,i)]-lam[(k,t,j)]) < 0:
            costo += (lam[(k,t,i)]-lam[(k,t,j)])
    d[(k,i,j)] = costo
    if (d[(k,i,j)] < 0):
        ePerKnapsack.append((k,i,j))
```

Knapsack

All'interno del metodo implementato per la risoluzione del problema dello zaino, l'unico aggiornamento riguarda la determinazione del valore delle x che, in caso di attivazione dell'arco, dovranno assumere un valore binario dipendente dalle penalità lagrangiane.

```
...
e[(k,i,j)] = 1.0
for t in NodiTarget[k]:
    if (lam[(k,t,i)]-lam[(k,t,j)]) < 0:
        x[(k,t,i,j)] = 1
...
```

Calcolo del Lower Bound Attuale

L'algoritmo per il calcolo del Lower Bound viene riscritto per riflettere la nuova funzione obiettivo lagrangiana (3.22), derivata dal rilassamento della terza formulazione.

```
Lb=0.0
for (i,j) in A:
    for k in K:
        Lb += C[i,j]*e[(k,i,j)]
        for t in NodiTarget[k]:
            Lb += (lam[(k,t,i)]-lam[(k,t,j)])*x[(k,t,i,j)]
for k in K:
    for t in NodiTarget[k]:
        Lb += P*(1-y[(k,t)])
        for i in N:
            Lb -= lam[(k,t,i)]*b[k][t][i]
```

Calcolo dello Step e Aggiornamento delle Penalità

Per quanto riguarda il calcolo dello step e il successivo aggiornamento delle penalità lagrangiane, la logica dell'algoritmo rimane invariata. L'unica modifica introdotta, riguarda la dimensione delle variabili coinvolte, le quali saranno indicizzate su tre parametri al posto dei precedenti due.

```
subGradienti = {(k,t,i):0.0 for k in K for t in NodiTarget[k] for i in N}
...
lam[(k,t,i)] = lam[(k,t,i)] + valoreDiCambio * subGradienti[(k,t,i)]
```

Main Loop

Il ciclo principale, invece, si modifica sia per gestire le chiamate alle varie funzioni descritte precedentemente sia per integrare la nuova variabile decisionale y all'interno della logica del problema.

```
y = {(k,t):0 for k in K for t in NodiTarget[k]}
lam = {(k,t,i):0.0 for k in K for t in NodiTarget[k] for i in N}
d = {(k,i,j):0 for k in K for (i,j) in A}
e = {(k,i,j):0 for k in K for (i,j) in A}
x = {(k,t,i,j):0.0 for k in K for t in NodiTarget[k] for (i,j) in A}
b = calcolaB(y)
ePerKnapsack = []
ultimiLowerBounds = [0]
Lb=0.0
alpha = 0.1
it_max=10000
alphaterazioni=0
iterazioni=0
UbEuristicoFormattato = TrovaSoluzioniEuristiche()
while abs(UbEuristicoFormattato-Lb)>1e-4 and iterazioni<it_max:
    iterazioni+=1
    ePerKnapsack=[]
    e = {(k,i,j):0 for k in K for (i,j) in A}
    x = {(k,t,i,j):0 for k in K for t in NodiTarget[k] for (i,j) in A}
    CalcolaY(lam, y)
    b = calcolaB(y)
    CalcolaCostiPenalizzati(lam,d,ePerKnapsack)
    KnapSack(ePerKnapsack,d,e,x,lam)
    Lb = calcolaMin(lam,b,y,e,x)
    ultimiLowerBounds.append(Lb)
    ...
    aggiornaLambda(lam,ultimiLowerBounds,UbEuristicoFormattato,alpha,x,b)
```

Cplex

Nel caso della terza formulazione, all'interno dell'ambiente Cplex, si è scelto di modificare la gestione delle variabili in modo da preservare la linearità e la leggibilità del codice. I due cambiamenti principali riguardano l'integrazione della b all'interno del vincolo, mediante l'uso di condizioni, e la creazione di una struttura KT contenente le combinazioni di tutte le informazioni e i relativi target.

```
minimize
sum(k in K, <i,j> in A) C[<i,j>] * e[k][<i,j>] +
sum(k in K, t in NodiTarget[k]) P * (1 - y[k][t]);

subject to {

    forall(<k, t> in KT, i in N) {
        sum(j in Lt[i]) x[<k,t>][<i,j>] - sum(j in Lr[i]) x[<k,t>][<j,i>] ==
        (i == NodoOrigine[k] ? y[k][t] : (i == t ? -y[k][t] : 0));
    }

    forall(<i,j> in A)
    sum(k in K) a[k] * e[k][<i,j>] <= u[<i,j>];

    forall(<k, t> in KT, <i, j> in A) {
        x[<k,t>][<i,j>] <= e[k][<i,j>];
    }

}
```

Python API

L'integrazione in linguaggio Python tramite l'API è stata sviluppata in modo da rispecchiare le modifiche effettuate precedentemente nell'ambiente Cplex. In primo luogo si creano le variabili decisionali e di supporto, garantendo nella definizione il rispetto dei loro vincoli associati (3.26) (3.27) (3.28).

```
modello = cplex.Cplex()
modello.objective.set_sense(modello.objective.sense.minimize)
KT = [(k, t) for k in K for t in NodiTarget[k]]
x = [(k,t,i,j) for (k, t) in KT for (i,j) in A]
modello.variables.add(names=[f"x_{k}_{t}_{i}_{j}" for (k,t,i,j) in x],
lb=[0]*len(x), types=["I"]*len(x))
e = [(k,i,j) for k in K for (i,j) in A]
modello.variables.add(names=[f"e_{k}_{i}_{j}" for (k,i,j) in e],
lb=[0]*len(e), ub=[1]*len(e), types=["B"]*len(e))
y = [(k,t) for (k, t) in KT]
modello.variables.add(names=[f"y_{k}_{t}" for (k,t) in y],
lb=[0]*len(y), ub=[1]*len(y), types=["B"]*len(y))
```

In secondo luogo, si definiscono la funzione obiettivo e i vincoli della formulazione, integrando la b e sfruttando la struttura KT appena creata. Una particolarità nell'implementazione della funzione obiettivo riguarda l'introduzione di un offset da aggiungere al calcolo, ovvero di un valore costante indipendente dalle variabili decisionali. In questo caso specifico, si tratta del valore P moltiplicato per il numero di combinazioni di KT .

Funzione Obiettivo (3.22):

```
temp_X = []
temp_Coeffs=[]
for k in K:
    for (i,j) in A:
        temp_Coeffs.append(C[i,j])
        temp_X.append(f"e_{k}-{i}-{j}")
for (k,t) in KT:
    temp_Coeffs.append(-P)
    temp_X.append(f"y_{k}-{t}")
modello.objective.set_linear(list(zip(temp_X,temp_Coeffs)))
offset = P * len(KT)
modello.objective.set_offset(offset)
```

Primo vincolo (3.23)

```
for (k,t) in KT:
    for i in N:
        temp_X = []
        temp_Coeffs=[]
        for j in Lt[i]:
            if (i,j) in A:
                temp_X.append(f"x_{k}-{t}-{i}-{j}")
                temp_Coeffs.append(1)
        for j in Lr[i]:
            if (j,i) in A:
                temp_X.append(f"x_{k}-{t}-{j}-{i}")
                temp_Coeffs.append(-1)
        if i==NodoOrigine[k]:
            temp_X.append(f"y_{k}-{t}")
            temp_Coeffs.append(-1)
        if i == t:
            temp_X.append(f"y_{k}-{t}")
            temp_Coeffs.append(1)
        modello.linear_constraints.add
        (lin_expr=[cplex.SparsePair(temp_X,temp_Coeffs)], senses=["E"], rhs=[0])
```

Secondo vincolo (3.24):

```
for (i,j) in A:
    temp_X = []
    temp_Coeffs=[]
```

```

for k in K:
    temp_X.append(f"e_{k}_{i}_{j}")
    temp_Coeffs.append(a[k])
    modello.linear_constraints.add
    (lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["L"], rhs=[u[i, j]])

```

Terzo vincolo (3.25):

```

for (k, t) in KT:
    for (i, j) in A:
        temp_X = []
        temp_Coeffs = []
        temp_X.append(f"x_{k}_{t}_{i}_{j}")
        temp_Coeffs.append(1)
        temp_X.append(f"e_{k}_{i}_{j}")
        temp_Coeffs.append(-1)
        modello.linear_constraints.add
        (lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["L"], rhs=[0])

```

5 Analisi dei Risultati

In questo capitolo si approfondisce l'applicazione pratica degli algoritmi sviluppati, con l'obiettivo primario di valutarne l'efficacia e la correttezza all'interno di diversi scenari di simulazione. Le istanze di test sono costruite per simulare una rete Publish/Subscribe, nella quale bisogna inviare simultaneamente più informazioni con la possibilità di target multipli e di nodi irraggiungibili. Al fine di una valutazione, la soluzione viene confrontata con l'ottimo calcolata direttamente nell'ambiente Cplex e la distanza tra le due rappresenta una metrica fondamentale e sufficiente per determinare l'ottimalità dell'algoritmo. La generazione degli scenari segue due modalità: una manuale per analizzare in modo mirato i casi limite e le potenziali criticità e una casuale per verificare la correttezza in contesti non preconfigurati. **TODO**

5.1 Scenari e Test

Durante l'analisi dei vari test vengono raccolte statistiche importanti al fine di valutare l'andamento e l'efficienza dell'algoritmo. Come misura principale, si adotta la distanza dall'ottimo e come indicatori secondari il tempo di esecuzione e le iterazioni. Questi due parametri non sono valutabili per tutti gli algoritmi e molti degli algoritmi soprattutto quelli Python API hanno un limite definito dalla libreria su quante variabili possono gestire quindi si andranno a perdere nelle valutazioni. Infine, in chiusura alla sezione dei test viene presentata una **TODO**

5.1.1 Primo Scenario

La prima istanza di test è una rete semplice con un totale di 4 nodi. Le connessioni sono effettuate in modo preciso rendendo i percorsi più brevi di soli due archi, tuttavia, i costi di questi collegamenti sono intenzionalmente elevati. L'obiettivo è vedere una scelta da parte degli algoritmi di un percorso più lungo al fine di ridurre i costi complessivi.

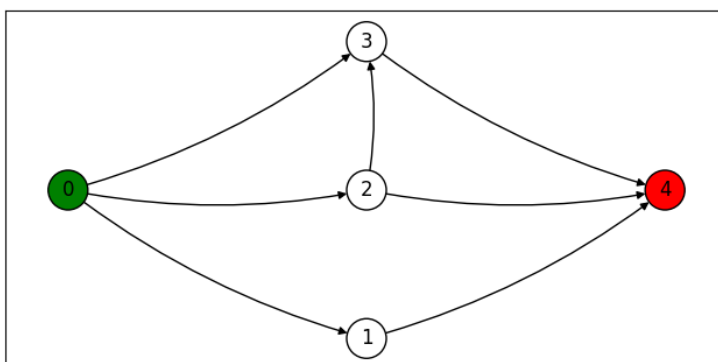


Figura 5.1: Foto rete del primo scenario.

Dati Informativi

La rete deve trasportare una informazione 0 dal nodo publisher 0 al subscriber 4 potendo passare da 3 nodi brokers. I costi dei collegamenti $0 \rightarrow 2 \rightarrow 3 \rightarrow 4$ sono molto ridotti.

Risoluzione

Inserendo il nostro scenario all'interno dei vari algoritmi si possono analizzare i risultati.

Algoritmo	Soluzione	Tempo (secondi)	Iterazioni
Prima Formulazione Cplex	7	/	/
Prima Formulazione Python API	7	0.0115	/
Seconda Formulazione Lagrangiana	7	0.0173	208
Seconda Formulazione Cplex	7	/	/
Seconda Formulazione Python API	7	0.0580	/
Terza Formulazione Lagrangiana	7	0.0120	120
Terza Formulazione Cplex	7	/	/
Terza Formulazione Python API	7	0.0481	/

Tabella 5.1: Panoramica dei risultati dei vari algoritmi per questo scenario.

Come emerge dai risultati, nonostante la prima formulazione e le successive due adottino misure differenti per il calcolo del costo del percorso ottimo, hanno tutte restituito la medesima soluzione con tempi molto ridotti. Tale soluzione, inoltre, risulta ottima per ciascun algoritmo sviluppato, in quanto coincide con il risultato restituito dagli algoritmi Cplex. Le soluzioni tra le formulazioni coincidono in quanto avendo una sola informazione la x si comporta come la e .

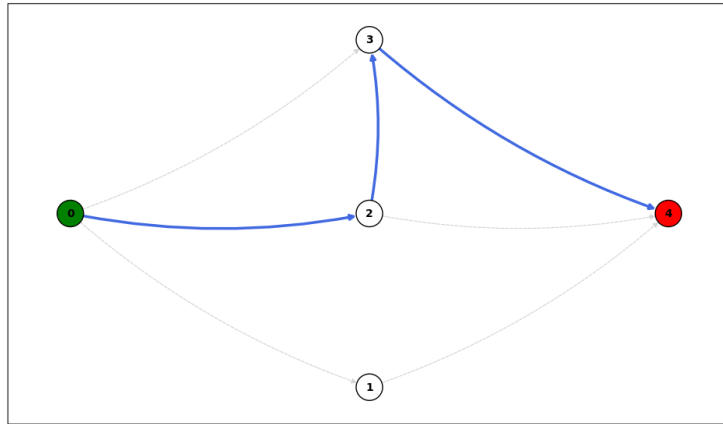


Figura 5.2: Percorso ottimo disegnato sulla base del risultato della terza formulazione.

5.1.2 Secondo Scenario

Il secondo scenario di test viene utilizzato per analizzare il comportamento dell'algoritmo in una rete densamente interconnessa. L'obiettivo è verificare la correttezza dell'algoritmo nella scelta del percorso ottimo in un contesto con molteplici cammini alternativi.

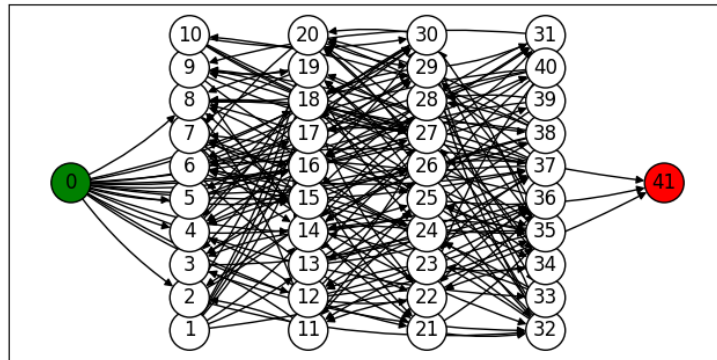


Figura 5.3: Secondo scenario.

Dati Informativi

La rete deve trasportare una informazione 0 dal nodo publisher 0 al subscriber 41 potendo passare da 40 nodi brokers differenti utilizzando 201 possibili archi distinti. La peculiarità dello scenario risiede nell'elevato numero di archi all'interno del grafo, progettati per indurre una forte ridondanza e una molteplicità di cammini alternativi.

Risoluzione

Algoritmo	Soluzione	Tempo (secondi)	Iterazioni
Prima Formulazione Cplex	113	/	/
Prima Formulazione Python API	113	0.0568	/
Seconda Formulazione Lagrangiana	113	0.9658	2700
Seconda Formulazione Cplex	113	/	/
Seconda Formulazione Python API	113	0.0500	/
Terza Formulazione Lagrangiana	113	0.8932	1640
Terza Formulazione Cplex	113	/	/
Terza Formulazione Python API	113	0.0549	/

Tabella 5.2: Panoramica dei risultati dei vari algoritmi per questo scenario.

Come emerge dalle soluzioni, tutti gli algoritmi hanno trovato l'ottimo con tempi relativamente brevi, tuttavia, si rileva un peggioramento delle tempistiche rispetto al primo scenario, da associare sia all'aumento dei percorsi da controllare tramite l'algoritmo di Upper Bound, sia all'aumento delle iterazioni dei cicli, i quali dipendono dal numero di archi e nodi. Analogamente a quanto osservato nel primo scenario, l'uguaglianza tra le soluzioni delle diverse formulazioni dipende dal comportamento della variabile x in scenari con una sola informazione da trasmettere.

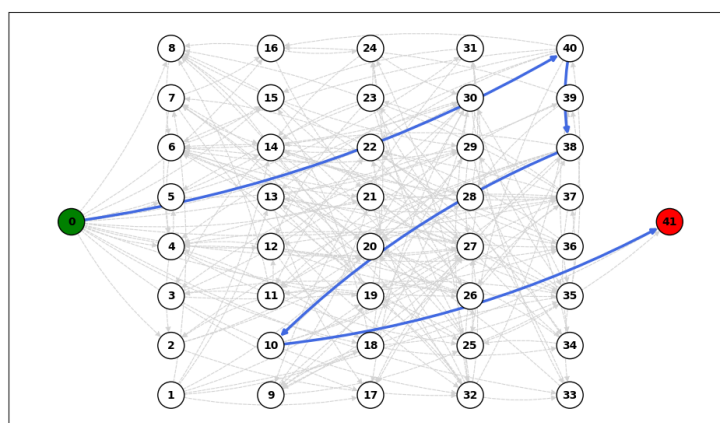


Figura 5.4: Percorso ottimo disegnato sulla base del risultato della terza formulazione.

5.1.3 Terzo Scenario

Il terzo scenario di test introduce una situazione di congestione della rete in cui tre informazioni, di cui una con due target, competono per arrivare ai nodi Subscriber. Le capacità degli archi sono state elaborate precisamente per indurre uno scenario di saturazione che colpisca il percorso ottimo costringendo agli algoritmi di dover usufruire di altri cammini per la trasmissione delle informazioni. Gli obiettivi di questo scenario consistono nel verificare la corretta saturazione della cammino ottimo e l'efficace scelta delle vie percorribile alternative.

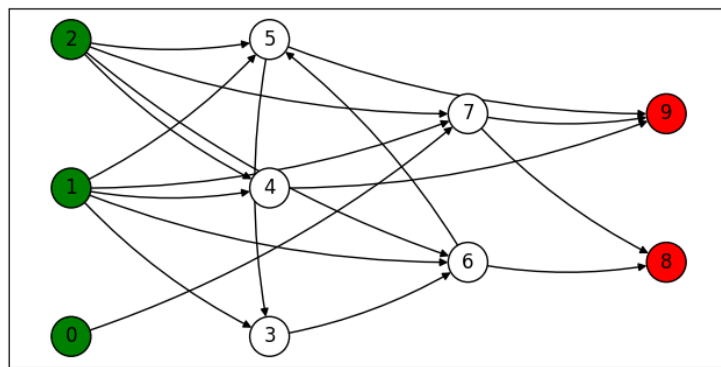


Figura 5.5: Terzo scenario

Dati Informativi

La rete deve trasportare tre informazioni 0, 1, 2 ai rispettivi nodi target. Nello specifico l'informazione 0 parte dal publisher 1 e deve raggiungere i target 8 e 9, l'informazione 1 parte dal publisher 2 e deve raggiungere il target 9 e, infine, l'informazione 2 parte dal publisher 0 e deve raggiungere il target 9. Il percorso ottimo per il raggiungimento del nodo 9 prevede il transito attraverso il nodo 7, tuttavia, la sua capacità è stata intenzionalmente limitata per simulare una saturazione.

Risoluzione

Algoritmo	Soluzione	Tempo (secondi)	Iterazioni
Prima Formulazione Cplex	229	/	/
Prima Formulazione Python API	229	0.0799	/
Seconda Formulazione Lagrangiana	233.73	2.0903	10000
Seconda Formulazione Cplex	234	/	/
Seconda Formulazione Python API	234	0.0536	/
Terza Formulazione Lagrangiana	234	0.2831	942
Terza Formulazione Cplex	234	/	/
Terza Formulazione Python API	234	0.0818	/

Tabella 5.3: Panoramica dei risultati dei vari algoritmi per questo scenario.

Dall'analisi delle soluzioni restituite dagli algoritmi emerge una mancata convergenza della seconda formulazione in modalità lagrangiana entro il limite massimo di iterazioni prefissate. Andando nel dettaglio, si nota che, mentre l'Upper Bound restituisce il risultato ottimo, il valore della soluzione lagrangiana si ferma a 233.73. Al contrario, la terza formulazione mostra un'eccellente efficienza computazionale, convergendo all'ottimo in tempi e iterazioni ridotte. Controllando la soluzione ottima, si riscontra il comportamento ricercato dallo scenario, ovvero la saturazione del cammino dal nodo 7 a 9, che costringe l'informazione in partenza dal nodo 2 a passare dal nodo 4 per raggiungere il nodo 9, nonostante il tragitto sia più costoso.

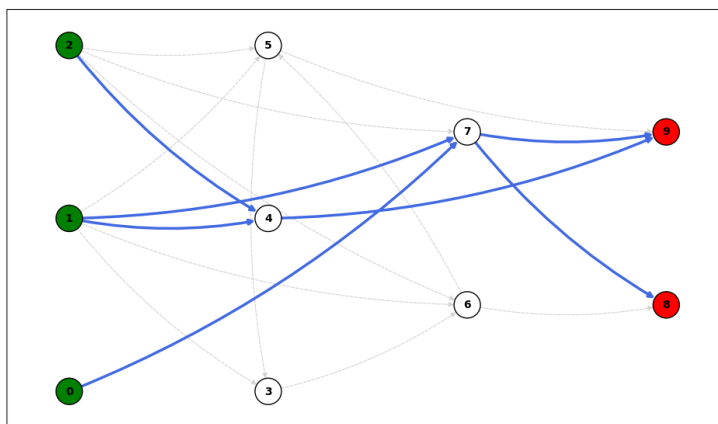


Figura 5.6: Percorso ottimo disegnato sulla base del risultato della terza formulazione.

5.1.4 Quarto Scenario

Il quarto scenario di test rappresenta una rete caratterizzata dalla saturazione dei percorsi ottimi per il raggiungimento di un nodo target. L'obiettivo dello scenario è di valutare la capacità dell'algoritmo derivante dalla terza formulazione di selezionare opportunamente quali informazioni instradare fino a destinazione e quali invece escludere, verificando parallelamente l'efficacia dell'algoritmo nella scelta dei percorsi ottimi.

5.1.5 Caso 5

Nel quinto caso di test si iniziano a controllare le criticità dell'algoritmo ovvero come si comporta con un elevato numero di Broker e Archi. In questo caso la foto non dà un'idea chiara del grafo quindi è inutile Test con nodi con singolo target e test con nodi con più target. Per questo test prendo un caso semplice come il primo e continuo ad aggiungere nodi super collegati che non verranno mai scelti solo per determinare se l'algoritmo continua a produrre il risultato ottimo e di quanto peggiora il tempo per iterazione.

5.1.6 Caso 6

Nel sesto caso di test si iniziano a controllare le criticità dell'algoritmo ovvero come si comporta con un elevato numero di target da raggiungere per informazione.

5.2 Analisi generale con grafici istogrammi e tabella con l'ottimo per ogni algoritmo per ogni scenario

5.3 Mostrare come cambia la convergenza al cambiare di alpha

5.4 Mostrare come cambia più soluzioni euristiche andiamo a prendere, cambiamo i valori dell'euristico

5.5 Soluzioni parziali

5.5.1 Caso 1

K= [0, 1, 2, 3, 4] N= [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35] A= [(0, 25), (1, 17), (1, 19), (2, 1), (2, 5), (3, 4), (3, 16), (4, 18), (4, 3), (5, 27), (6, 11), (7, 21), (8, 29), (8, 9), (9, 19), (10, 4), (11, 30), (11, 29), (12, 15), (12, 19), (13, 22), (13, 18), (14, 2), (15, 28), (15, 3), (16, 14), (16, 29), (17, 4), (18, 15), (18, 23), (19, 12), (20, 7), (20, 16), (21, 4), (22, 24), (22, 3), (23, 21), (23, 13), (24, 10), (24, 18), (25, 24), (26, 17), (26, 29), (27, 11), (27, 23), (28, 20), (28, 9), (29, 1), (30, 23), (11, 31), (23, 31), (14, 32), (29, 32), (1, 33), (7, 34), (23, 34), (29, 35), (15, 35)]
Subscribers= [31, 32, 33, 34, 35] Brokers= [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30] Publishers= [0]
NodoOrigine= 0: 0, 1: 0, 2: 0, 3: 0, 4: 0 NodiTarget= 0: [31, 32, 35, 34, 33], 1: [35, 34, 31], 2: [33, 34, 31, 32], 3: [34, 33, 31], 4: [35] a= 0: 2, 1: 4, 2: 6, 3: 1, 4: 8
C= (0, 25): 26, (1, 17): 58, (1, 19): 12, (2, 1): 20, (2, 5): 49, (3, 4): 1, (3, 16): 53, (4, 18): 14, (4, 3): 89, (5, 27): 30, (6, 11): 45, (7, 21): 94, (8, 29): 38, (8, 9): 76, (9, 19): 81, (10, 4): 95, (11, 30): 54, (11, 29): 51, (12, 15): 73, (12, 19): 48, (13, 22): 17, (13, 18): 3, (14, 2): 66, (15, 28): 39, (15, 3): 98, (16, 14): 9, (16, 29): 51, (17, 4): 86, (18, 15): 92, (18, 23): 58, (19, 12): 56, (20, 7): 21, (20, 16): 90, (21, 4): 48, (22, 24): 27, (22, 3): 81, (23, 21): 99, (23, 13): 5, (24, 10): 57, (24, 18): 21, (25, 24): 79, (26, 17): 30, (26, 29): 59, (27, 11): 41, (27, 23): 1, (28, 20): 84, (28, 9): 91, (29, 1): 99, (30, 23): 82, (11, 31): 18, (23, 31): 8, (14, 32): 73, (29, 32): 29, (1, 33): 62, (7, 34): 91, (23, 34): 74, (29, 35): 72, (15, 35): 80
u= (0, 25): 2, (1, 17): 11, (1, 19): 7, (2, 1): 5, (2, 5): 15, (3, 4): 7, (3, 16): 1, (4, 18): 1, (4, 3): 15, (5, 27): 12, (6, 11): 1, (7, 21): 15, (8, 29): 14, (8, 9): 10, (9, 19): 2, (10, 4): 1, (11, 30): 14, (11, 29): 3, (12, 15): 8, (12, 19): 14, (13, 22): 14,

(13, 18): 2, (14, 2): 14, (15, 28): 4, (15, 3): 1, (16, 14): 13, (16, 29): 2, (17, 4): 3, (18, 15): 14, (18, 23): 8, (19, 12): 15, (20, 7): 2, (20, 16): 13, (21, 4): 10, (22, 24): 12, (22, 3): 6, (23, 21): 3, (23, 13): 3, (24, 10): 5, (24, 18): 12, (25, 24): 10, (26, 17): 10, (26, 29): 14, (27, 11): 9, (27, 23): 9, (28, 20): 14, (28, 9): 9, (29, 1): 1, (30, 23): 14, (11, 31): 4, (23, 31): 15, (14, 32): 3, (29, 32): 7, (1, 33): 5, (7, 34): 4, (23, 34): 9, (29, 35): 5, (15, 35): 14 3105

5.6 Caso rotto e ricalcolo

K= [0]

N= [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

A= [(0, 3), (0, 1), (0, 2), (0, 6), (0, 4), (0, 7), (0, 5), (1, 2), (2, 5), (2, 4), (3, 7), (3, 2), (3, 6), (5, 3), (5, 1), (6, 3), (7, 3), (7, 2), (1, 8), (3, 8), (2, 8), (6, 8), (7, 8), (6, 9)]

Subscribers= [8, 9]

Brokers= [1, 2, 3, 4, 5, 6, 7]

Publishers= [0]

NodoOrigine= 0: 0, 1: 0, 2: 0

NodiTarget= 0: [9, 8], 1: [8], 2: [8, 9]

a= 0: 5, 1: 2, 2: 4

C= (0, 3): 8, (0, 1): 9, (0, 2): 2, (0, 6): 1, (0, 4): 2, (0, 7): 7, (0, 5): 8, (1, 2): 8, (2, 5): 7, (2, 4): 1, (3, 7): 5, (3, 2): 1, (3, 6): 1, (5, 3): 5, (5, 1): 3, (6, 3): 9, (7, 3): 9, (7, 2): 7, (1, 8): 7, (3, 8): 3, (2, 8): 7, (6, 8): 1, (7, 8): 1, (6, 9): 10

u= (0, 3): 16, (0, 1): 11, (0, 2): 20, (0, 6): 20, (0, 4): 13, (0, 7): 23, (0, 5): 7, (1, 2): 1, (2, 5): 16, (2, 4): 18, (3, 7): 25, (3, 2): 12, (3, 6): 6, (5, 3): 5, (5, 1): 9, (6, 3): 9, (7, 3): 12, (7, 2): 4, (1, 8): 21, (3, 8): 24, (2, 8): 8, (6, 8): 25, (7, 8): 1, (6, 9): 13

5.7 caso x

5.8 Modifica di alpha

5.9 Modifica di variabile x

6 Conclusioni

Dijkstra si basa sull'ottimo prima per andare avanti, può essere migliorato